

UNIVERSITÀ DEGLI STUDI DI ...

Dipartimento di ...

Corso di Laurea in ...

PCB-CTF

Piattaforma Web Interattiva per la Formazione
sulla Sicurezza Hardware in Formato CTF

Relatore

Prof. ...

Candidato

Davide Leone

Matricola: ...

Anno Accademico 2025/2026

Indice

Introduzione	1
1 Analisi dei requisiti	4
1.1 Contesto operativo e attori del sistema	4
1.2 Requisiti funzionali	5
1.2.1 Simulazione del circuito stampato	5
1.2.2 Strumenti di misura	5
1.2.3 Terminale simulato	6
1.2.4 Strumenti personalizzati	7
1.2.5 Struttura dell'esercizio	7
1.2.6 Authoring e configurazione	8
1.2.7 Lancio automatico del terminale	8
1.3 Requisiti non funzionali	9
1.3.1 Accessibilità e distribuzione	9
1.3.2 Usabilità	9
1.3.3 Prestazioni	9
1.3.4 Manutenibilità e estensibilità	10
1.3.5 Sicurezza	10
1.4 Casi d'uso principali	10
1.4.1 Casi d'uso dello studente	10
1.4.2 Casi d'uso dell'autore	11
1.5 Vincoli e assunzioni	12
2 Progettazione dell'architettura	14
2.1 Scelte tecnologiche	14
2.2 Architettura generale del sistema	15
2.3 Modello dei dati	16
2.3.1 Struttura dell'esercizio	16
2.3.2 Il sistema unificato dei pin	17

2.3.3	Il sistema di coordinate	17
2.4	Gestione dello stato applicativo	18
2.4.1	Architettura a store indipendenti	18
2.4.2	Coordinazione tra store	19
2.4.3	Persistenza dello stato	19
2.4.4	Migrazione dei dati	20
2.5	Progettazione del sistema terminale	20
2.5.1	Principi di progettazione	20
2.5.2	Architettura modulare	21
2.5.3	Pipeline di esecuzione dei comandi	21
2.5.4	Il filesystem simulato	22
2.5.5	La sequenza di boot	22
2.5.6	Il sistema di flag	23
2.6	Flusso dati tra authoring e simulazione	23
2.6.1	Il sistema di preset	24
3	Implementazione	25
3.1	Il visualizzatore PCB interattivo	25
3.1.1	Sistema di overlay e conversione delle coordinate	25
3.1.2	Meccanismo di snap	26
3.1.3	Rendering dei fili	26
3.1.4	La lente di ingrandimento	27
3.2	Strumenti di analisi hardware	28
3.2.1	Il multimetro	28
3.2.2	L'adattatore UART	28
3.2.3	Il firmware dumper SPI	29
3.3	Il terminale simulato	31
3.3.1	Il loader della configurazione	31
3.3.2	L'esecutore dei comandi	32
3.3.3	I comandi built-in	32
3.3.4	La sequenza di boot	33
3.3.5	Persistenza dello stato del terminale	34
3.4	Il sistema di strumenti personalizzati	34
3.4.1	Modello dati	34
3.4.2	Rendering nel simulatore	35
3.4.3	Authoring degli strumenti personalizzati	35
3.5	Il pannello di authoring	36
3.5.1	Organizzazione dell'interfaccia	36

3.5.2	Il canvas interattivo	37
3.5.3	L'editor del terminale	37
3.6	API e persistenza dei dati	38
3.6.1	Architettura delle API	38
3.6.2	Strategia di persistenza	39
3.6.3	Sincronizzazione real-time	40
3.7	Il sistema di completamento dell'esercizio	40
3.7.1	Flussi di completamento degli obiettivi	40
3.7.2	Reset dello stato alla transizione tra step	41
3.7.3	Progressione della flag	42
3.7.4	Il dialogo di completamento	42
4	Caso di studio	43
4.1	Progettazione dell'esercizio	43
4.1.1	Struttura dell'esercizio	43
4.1.2	Configurazione del terminale	44
4.1.3	Configurazione degli strumenti	45
4.2	Walkthrough dello studente	45
4.2.1	Fase 1 — Ricognizione del PCB	46
4.2.2	Fase 2 — Connessione UART	47
4.2.3	Fase 3 — Esplorazione del terminale	47
4.3	Configurazione lato autore	49
4.3.1	Creazione dell'esercizio	49
4.3.2	Configurazione del terminale	49
4.3.3	Salvataggio come preset	50
5	Validazione e discussione	51
5.1	Criteri di valutazione	51
5.1.1	Copertura dei concetti didattici	51
5.1.2	Fedeltà della simulazione	52
5.1.3	Espressività del sistema di authoring	53
5.2	Confronto con soluzioni esistenti	54
5.3	Limitazioni	55
5.4	Sviluppi futuri	55
	Conclusioni	57

A	Struttura JSON di un esercizio completo	59
A.1	Schema dell'esercizio (Exercise)	59
A.2	Tipi enumerati	68
A.2.1	ObjectiveType	68
A.2.2	Tool	68
A.2.3	UartRole	69
A.2.4	SpiRole	69
A.3	Sistema di coordinate	69
A.4	Esempio completo	70
B	Schema della configurazione terminale	74
B.1	Struttura principale (TerminalConfig)	74
B.2	Configurazione del tab (TabConfig)	75
B.2.1	Stadi speciali della sequenza di boot	76
B.3	Definizione di un comando (CommandDefinition)	77
B.3.1	Vincoli del comando (CommandConstraints)	79
B.3.2	Tipi di output (CommandOutput)	80
B.3.3	Controllo delle condizioni (ConditionCheck)	82
B.4	Struttura del filesystem (FilesystemStructure)	83
B.4.1	Processo di normalizzazione	85
B.5	Sistema di flag (FlagSystem)	85
B.6	Contesto di esecuzione (CommandContext)	86
B.7	Risultato dell'esecuzione (CommandExecutionResult)	86
B.8	Esempio: configurazione del tab UART Console	87

Introduzione

Contesto

La sicurezza dei sistemi embedded pervade ormai ogni aspetto della vita quotidiana. Router domestici, telecamere di sorveglianza, dispositivi medici, sistemi di controllo industriale: tutti condividono una caratteristica comune, ovvero la presenza di hardware programmabile il cui firmware può contenere vulnerabilità critiche. Gli attacchi a dispositivi IoT sono in costante crescita, e i report annuali delle principali aziende di cybersecurity documentano un panorama preoccupante: credenziali di default non modificabili, comunicazioni in chiaro, backdoor intenzionali, e firmware privo di meccanismi di aggiornamento sicuro.

In questo scenario, la capacità di analizzare un dispositivo a livello hardware — ispezionarne il circuito stampato, identificare le interfacce di debug, estrarre e analizzare il firmware — rappresenta una competenza sempre più richiesta nel campo della sicurezza informatica. Tuttavia, la formazione in questo ambito presenta sfide specifiche che la distinguono dalla tradizionale formazione sulla sicurezza software.

Il problema

L'insegnamento delle tecniche di analisi hardware soffre di una barriera d'ingresso significativamente più alta rispetto alla controparte software. Mentre uno studente può iniziare a praticare il penetration testing web con un browser e una macchina virtuale, l'analisi hardware richiede attrezzature fisiche — multimetri, analizzatori logici, adattatori UART e SPI, programmatori di memorie flash — il cui costo complessivo non è trascurabile. A questo si aggiunge la necessità di disporre del dispositivo target stesso, con il rischio concreto di danneggiarlo durante le fasi di apprendimento.

Le piattaforme CTF (Capture The Flag) hanno rivoluzionato la formazione nella sicurezza software, offrendo ambienti sicuri e gamificati dove gli studenti possono esercitarsi senza rischi. Tuttavia, l'estensione di questo modello all'hardware si è

rivelata problematica. Le soluzioni esistenti ricadono tipicamente in due categorie: le piattaforme puramente software, che propongono sfide di reverse engineering di firmware senza interazione con l'hardware fisico, e i laboratori remoti, che offrono accesso a dispositivi reali ma con costi infrastrutturali elevati e scalabilità limitata.

Manca, ad oggi, una soluzione intermedia che consenta di simulare in modo fedele l'interazione fisica con un circuito stampato — l'uso degli strumenti di misura, il collegamento delle sonde, l'estrazione del firmware — in un ambiente completamente virtuale, accessibile da un semplice browser.

Obiettivo della tesi

Il presente lavoro propone la progettazione e l'implementazione di **PCB-CTF**, una piattaforma web interattiva per la formazione sulla sicurezza hardware in formato Capture The Flag. La piattaforma consente agli studenti di interagire con un circuito stampato virtuale attraverso strumenti simulati — multimetro digitale, adattatore UART, programmatore SPI, terminale embedded — replicando il flusso di lavoro di un'analisi hardware reale, dal riconoscimento visivo dei componenti fino all'estrazione e all'analisi del firmware.

L'obiettivo non è sostituire l'esperienza pratica su hardware reale, ma fornire un complemento didattico che permetta di acquisire le competenze concettuali e metodologiche prima di accedere al laboratorio fisico, riducendo la curva di apprendimento e il rischio di danneggiamento delle attrezzature.

La piattaforma si articola in due modalità operative: un **simulatore** per gli studenti, dove il percorso didattico è strutturato in step con obiettivi progressivi e flag da scoprire, e un **pannello di authoring** per i docenti, che consente di progettare esercizi personalizzati senza modificare il codice sorgente, utilizzando un editor visuale e un sistema di configurazione dichiarativa.

Contributo

I principali contributi di questo lavoro sono:

1. La progettazione di un'architettura web che consente la simulazione interattiva di strumenti di misura hardware (multimetro, sonde UART, programmatore SPI) con feedback visivo realistico, interamente lato client.

2. L'implementazione di un terminale simulato configurabile che riproduce il comportamento di un sistema embedded — sequenza di boot, filesystem, comandi — senza eseguire codice sul server, garantendo sicurezza e determinismo.
3. La realizzazione di un sistema di authoring che consente ai docenti di creare esercizi completi attraverso un'interfaccia grafica, definendo il layout del PCB, gli strumenti disponibili, la struttura del terminale e il percorso didattico, senza richiedere competenze di programmazione.
4. La validazione della piattaforma attraverso un caso di studio basato su un dispositivo reale (TP-Link WR841N), che dimostra la capacità del sistema di replicare un percorso di analisi hardware completo.

Struttura del documento

La tesi è organizzata come segue.

Il **Capitolo 1** offre una panoramica sullo stato dell'arte nella sicurezza hardware, sulle piattaforme CTF esistenti, sugli approcci alla simulazione per la didattica e sulle tecnologie web utilizzate.

Il **Capitolo 2** presenta l'analisi dei requisiti funzionali e non funzionali della piattaforma, i casi d'uso principali e i vincoli progettuali.

Il **Capitolo 3** descrive la progettazione dell'architettura: le scelte tecnologiche, il modello dei dati, la gestione dello stato applicativo, la progettazione del sistema terminale e il flusso di dati tra le modalità di authoring e simulazione.

Il **Capitolo 4** dettaglia l'implementazione dei componenti principali: il visualizzatore PCB interattivo, gli strumenti di analisi hardware, il terminale simulato, il sistema di strumenti personalizzati, il pannello di authoring e le API di persistenza.

Il **Capitolo 5** presenta un caso di studio completo basato sull'analisi del router TP-Link WR841N, illustrando il percorso dello studente e la configurazione lato autore.

Il **Capitolo 6** discute i risultati ottenuti, confronta la piattaforma con le soluzioni esistenti, ne analizza le limitazioni e propone direzioni di sviluppo futuro.

Chiudono il lavoro le **conclusioni**, che sintetizzano il contributo e le riflessioni finali.

Capitolo 1

Analisi dei requisiti

1.1 Contesto operativo e attori del sistema

Prima di procedere con l'elicitazione dei requisiti, è necessario definire con chiarezza il contesto d'uso della piattaforma e gli attori coinvolti.

La piattaforma PCB-CTF è concepita per l'uso in ambito accademico e formativo, dove un docente — o più in generale un autore di contenuti — predispone un esercizio di analisi hardware che uno o più studenti dovranno risolvere. I due attori principali operano in momenti e modalità distinte:

- Lo **studente** accede al simulatore via browser, visualizza il circuito stampato, utilizza gli strumenti virtuali messi a disposizione e risolve progressivamente gli obiettivi proposti. Il suo percorso è guidato da istruzioni testuali e suggerimenti, e il completamento di ciascuna fase è attestato dalla scoperta di un frammento della flag finale.
- L'**autore** (docente, tutor, o progettista dell'esercizio) accede al pannello di configurazione, dove dispone i componenti sulla fotografia del PCB, definisce i punti di misura, configura il comportamento del terminale e struttura la sequenza didattica in step e obiettivi. Il suo lavoro produce una configurazione JSON che il simulatore consumerà.

Non è previsto, nella versione attuale, un terzo attore con ruolo amministrativo: l'autore ha pieno controllo su tutte le funzionalità di configurazione. Questa semplificazione riflette lo scenario d'uso primario, dove il docente opera in autonomia sulla propria istanza della piattaforma.

1.2 Requisiti funzionali

I requisiti funzionali sono stati organizzati per area tematica, riflettendo le macro-funzionalità della piattaforma.

1.2.1 Simulazione del circuito stampato

RF-01 — Visualizzazione del PCB. Il sistema deve consentire la visualizzazione di una fotografia ad alta risoluzione di un circuito stampato, sulla quale vengono sovrapposti elementi interattivi (componenti cliccabili, pin, overlay degli strumenti). L'interfaccia deve adattarsi alle dimensioni della finestra del browser mantenendo le proporzioni degli elementi.

RF-02 — Identificazione dei componenti. Lo studente deve poter individuare e selezionare componenti hardware sul PCB attraverso il click su aree predefinite dall'autore. Il sistema deve fornire feedback visivo per distinguere componenti già identificati da quelli ancora da scoprire.

RF-03 — Strumento di ingrandimento. Il sistema deve fornire una lente di ingrandimento virtuale che consenta di esaminare i dettagli del PCB con un livello di zoom configurabile. La lente deve poter essere ancorata in una posizione fissa per liberare il cursore.

1.2.2 Strumenti di misura

RF-04 — Multimetro digitale. Il sistema deve simulare un multimetro digitale con due sonde (rossa e nera), modalità tensione e modalità resistenza. Il multimetro deve visualizzare valori realistici con rumore digitale simulato. Le letture devono rispettare le regole fisiche: in modalità tensione con due sonde, il display mostra la differenza di potenziale; in modalità resistenza, la somma delle resistenze.

RF-05 — Adattatore UART. Il sistema deve simulare un adattatore USB-to-serial con pin TX, RX e GND trascinabili. Il collegamento deve rispettare la logica del protocollo UART, in particolare la corrispondenza incrociata tra TX e RX. Il sistema deve validare in tempo reale la correttezza dei collegamenti senza rivelare esplicitamente la soluzione.

RF-06 — Strumento di firmware dump SPI. Il sistema deve simulare un programmatore SPI con sei sonde corrispondenti ai segnali del bus (VCC, GND,

CS, CLK, MOSI, MISO). Ogni sonda deve poter essere collegata solo al pin con il ruolo corrispondente. Al completamento delle connessioni, il sistema deve simulare il trasferimento del firmware con una barra di progresso e offrire il download di un file binario reale.

RF-07 — Connessione delle sonde. Tutti gli strumenti con sonde trascinabili devono implementare un meccanismo di aggancio automatico (snap) che faciliti il collegamento ai pin vicini al cursore. Il raggio di aggancio deve essere sufficientemente ampio da rendere l'interazione confortevole ma non così ampio da generare ambiguità quando i pin sono ravvicinati.

1.2.3 Terminale simulato

RF-08 — Emulazione del terminale. Il sistema deve fornire un terminale testuale interattivo che simuli un ambiente a riga di comando, senza eseguire effettivamente comandi sul server. Il terminale deve supportare comandi built-in (`ls`, `cd`, `cat`, `pwd`, `grep`, `find`, `clear`) e comandi personalizzati definiti dall'autore.

RF-09 — Filesystem simulato. Il terminale deve operare su un filesystem virtuale configurabile, con supporto per directory, file con contenuto testuale, permessi e proprietari. Il filesystem deve essere navigabile tramite i comandi standard.

RF-10 — Sequenza di boot. Il terminale deve poter simulare una sequenza di avvio del dispositivo, con stadi animati che riproducono il flusso di un boot reale (bootloader, kernel, login). Alcuni stadi devono prevedere interazione da parte dell'utente (inserimento credenziali).

RF-11 — Sistema di flag terminale. Il terminale deve implementare un meccanismo di scoperta progressiva delle flag: l'esecuzione di determinati comandi sblocca frammenti della flag, e l'interfaccia mostra in tempo reale il progresso raggiunto.

RF-12 — Tab multipli. Il terminale deve supportare tab multipli, ciascuno con il proprio contesto di esecuzione (filesystem, percorso corrente, cronologia). Questo consente di simulare scenari realistici in cui lo studente opera simultaneamente sul dispositivo target (via UART) e sulla propria macchina di analisi.

RF-13 — Vincoli sui comandi. Il sistema deve consentire all'autore di definire vincoli sull'esecuzione dei comandi: disponibilità limitata a specifiche directo-

ry, requisito di permessi, prerequisiti (comandi già eseguiti o flag già scoperte), e restrizione a specifici stadi di boot.

1.2.4 Strumenti personalizzati

RF-14 — Definizione di strumenti custom. L'autore deve poter definire strumenti personalizzati con sonde, modalità di misura e tipi di output configurabili. Gli strumenti devono supportare almeno le seguenti tipologie di output: numerico (lettura di valori dai pin), indicatore LED, stato di connessione, e dump firmware.

RF-15 — Integrazione nel simulatore. Gli strumenti personalizzati devono apparire nella sidebar del simulatore accanto agli strumenti built-in e devono implementare lo stesso meccanismo di drag-and-drop e snap delle sonde.

1.2.5 Struttura dell'esercizio

RF-16 — Organizzazione in step e obiettivi. L'esercizio deve essere strutturato in step sequenziali, ciascuno con uno o più obiettivi. Lo studente affronta gli step in ordine; all'interno di ogni step, gli obiettivi possono essere risolti nella sequenza prevista dall'autore.

RF-17 — Tipologie di obiettivi. Il sistema deve supportare almeno cinque tipologie di obiettivi: identificazione di componenti (click), condizioni su pin (valori di misura), connessione UART, interazione con il terminale (scoperta di flag), ed estrazione firmware SPI.

RF-18 — Flag progressiva. Il completamento di ogni step e obiettivo deve contribuire alla costruzione progressiva della flag finale dell'esercizio. L'interfaccia deve mostrare in tempo reale la flag parziale, con indicazione chiara dei frammenti ancora da scoprire.

RF-19 — Dialogo di completamento. Al termine dell'esercizio, il sistema deve mostrare un dialogo configurabile che può includere: la flag completa con possibilità di copia, un pulsante di redirect verso una piattaforma esterna, e un pulsante per il download di un file.

1.2.6 Authoring e configurazione

RF-20 — Editor visuale del PCB. L'autore deve poter posizionare componenti e pin sulla fotografia del PCB tramite interazione diretta (drag-and-drop), con supporto per zoom, rotazione e traslazione della vista.

RF-21 — Editor del terminale. Il sistema deve fornire un editor strutturato per la configurazione del terminale: definizione dei comandi (tipo, output, vincoli, effetti collaterali), organizzazione del filesystem ad albero, configurazione della sequenza di boot, e definizione delle parti della flag.

RF-22 — Anteprima del terminale. L'autore deve poter testare il terminale configurato direttamente dal pannello di authoring, senza necessità di passare al simulatore.

RF-23 — Sistema di preset. Il sistema deve consentire il salvataggio e il caricamento di configurazioni complete (esercizio + terminale) come preset riutilizzabili. Ogni preset deve essere autocontenuto, includendo una copia dell'immagine del PCB.

RF-24 — Configurazione degli strumenti built-in. L'autore deve poter configurare i parametri degli strumenti built-in: raggio e livello di zoom della lente, comportamento dell'adattatore UART (persistenza, associazione con il terminale), parametri del firmware dump (sonde, durata, file binario).

RF-25 — Gruppi di strumenti. L'autore deve poter definire gruppi di strumenti attivabili contemporaneamente, superando il vincolo predefinito di mutua esclusività.

1.2.7 Lancio automatico del terminale

RF-26 — Auto-lancio del terminale. Il terminale deve poter essere attivato automaticamente in risposta a eventi specifici: completamento di una connessione UART, completamento di un dump firmware, o attivazione di un obiettivo di tipo terminale. Il componente terminale da lanciare e il tab iniziale devono essere configurabili.

1.3 Requisiti non funzionali

1.3.1 Accessibilità e distribuzione

RNF-01 — Zero installazione. La piattaforma deve funzionare interamente nel browser, senza richiedere l'installazione di software aggiuntivo, plugin o estensioni lato studente. L'unico prerequisito deve essere un browser moderno con supporto JavaScript.

RNF-02 — Deployment semplificato. Il sistema deve produrre un artefatto di deployment autocontenuto (standalone), deployabile in ambienti containerizzati (Docker) senza dipendenze runtime esterne. Non deve essere richiesto un database o servizi esterni.

1.3.2 Usabilità

RNF-03 — Guidabilità. Ogni obiettivo deve essere accompagnato da un'istruzione testuale e da un suggerimento opzionale, in modo che lo studente possa procedere anche senza conoscenze pregresse specifiche sull'hardware in esame.

RNF-04 — Feedback visivo. Il sistema deve fornire feedback visivo immediato per tutte le interazioni: evidenziazione dei pin nell'area di snap, animazione dei fili durante il trascinamento, indicazione cromatica dello stato delle connessioni, barra di progresso durante il dump firmware.

RNF-05 — Responsività. L'interfaccia deve adattarsi a diverse risoluzioni e dimensioni dello schermo senza compromettere la funzionalità. Il sistema di coordinate normalizzate deve garantire il corretto posizionamento degli elementi interattivi indipendentemente dalla dimensione della finestra.

1.3.3 Prestazioni

RNF-06 — Esecuzione lato client. Tutte le interazioni dello studente — misurazione con il multimetro, connessione delle sonde, esecuzione dei comandi nel terminale — devono essere elaborate lato client, senza round-trip al server, per garantire latenza minima e responsività immediata.

RNF-07 — Indipendenza dal server durante la simulazione. Una volta caricata la configurazione iniziale, il simulatore deve funzionare senza ulteriori chia-

mate al server. La perdita di connettività non deve interrompere la sessione in corso.

1.3.4 Manutenibilità e estensibilità

RNF-08 — Tipizzazione statica. Il modello dati deve essere integralmente tipizzato con TypeScript, in modo da intercettare errori di struttura in fase di compilazione.

RNF-09 — Modularità del terminale. Il sistema terminale deve supportare la definizione di più componenti terminale indipendenti all'interno di un singolo esercizio, per consentire scenari con dispositivi multipli o con ambienti di analisi separati.

RNF-10 — Configurazione dichiarativa. Il comportamento del terminale — comandi, vincoli, output, effetti collaterali — deve essere definibile in modo dichiarativo tramite configurazione JSON, senza richiedere modifiche al codice sorgente per la creazione di nuovi esercizi.

1.3.5 Sicurezza

RNF-11 — Nessuna esecuzione server-side di comandi. Il terminale simulato non deve mai eseguire comandi reali sul sistema operativo del server. Ogni interazione deve essere gestita dal motore di esecuzione lato client, operando esclusivamente sulle strutture dati in memoria.

RNF-12 — Isolamento delle sessioni. Lo stato della sessione di uno studente deve essere completamente indipendente e non deve interferire con le sessioni di altri studenti o con la configurazione dell'autore.

1.4 Casi d'uso principali

Si descrivono di seguito i casi d'uso più rilevanti, organizzati per attore.

1.4.1 Casi d'uso dello studente

CU-01 — Esplorazione del PCB. Lo studente apre il simulatore, visualizza il circuito stampato e utilizza la lente di ingrandimento per esaminare i dettagli. Iden-

tifica i componenti hardware cliccando sulle aree corrispondenti. Al completamento di ogni identificazione, riceve un frammento della flag.

CU-02 — Misurazione elettrica. Lo studente seleziona il multimetro dalla sidebar e trascina le sonde sui punti di test del circuito. Il display mostra i valori di tensione o resistenza. L'obiettivo viene completato quando le sonde sono posizionate sui pin che soddisfano le condizioni definite dall'autore.

CU-03 — Connessione UART. Lo studente seleziona lo strumento delle sonde UART e collega i pin dell'adattatore ai pin del circuito, rispettando la logica di crossover del protocollo. Al completamento della connessione corretta, il terminale si apre automaticamente simulando una sessione seriale.

CU-04 — Estrazione firmware. Lo studente seleziona lo strumento di firmware dump e collega le sonde SPI ai pin corrispondenti sul PCB. Una volta completate le connessioni richieste, avvia il dump e scarica il file firmware al termine del trasferimento simulato.

CU-05 — Esplorazione del terminale. Lo studente interagisce con il terminale simulato: esplora il filesystem, esegue comandi, analizza file di configurazione, scopre credenziali e vulnerabilità. Ogni scoperta significativa sblocca un frammento della flag terminale.

CU-06 — Completamento dell'esercizio. Lo studente completa tutti gli step dell'esercizio, inserisce la flag finale nel dialogo di validazione, e riceve conferma del completamento. Il dialogo finale può offrire opzioni di redirect o download.

1.4.2 Casi d'uso dell'autore

CU-07 — Configurazione del PCB. L'autore carica la fotografia del circuito, posiziona i componenti cliccabili e definisce i pin di misura (con valori elettrici), i pin UART (con ruoli) e i pin SPI (con ruoli). Utilizza il canvas interattivo per il posizionamento preciso.

CU-08 — Definizione dell'esercizio. L'autore crea step sequenziali, ciascuno con obiettivi tipizzati. Per ogni obiettivo definisce istruzioni, suggerimenti, frammento di flag e condizioni di completamento specifiche per il tipo (click su componente, condizioni su pin, flag terminale, connessione UART, dump firmware).

CU-09 — Configurazione del terminale. L'autore crea uno o più componenti terminale, ciascuno con tab, comandi, filesystem, sequenza di boot e flag. Utilizza l'editor strutturato per definire il comportamento di ogni comando e l'anteprima integrata per verificare il risultato.

CU-10 — Gestione dei preset. L'autore salva la configurazione corrente come preset, la carica da un preset esistente, o aggiorna un preset precedentemente salvato. Il sistema traccia le modifiche non salvate rispetto al preset attivo.

CU-11 — Creazione di strumenti personalizzati. L'autore definisce strumenti personalizzati con sonde, modalità e tipi di output. Carica eventuali immagini per il corpo dello strumento e configura il comportamento di ogni sonda.

1.5 Vincoli e assunzioni

La progettazione ha tenuto conto di un insieme di vincoli e assunzioni derivanti dal contesto d'uso:

V-01 — Singolo utente simultaneo per istanza. La piattaforma è progettata per un uso single-user per istanza. Non è prevista la gestione di sessioni concorrenti con autenticazione: ogni studente opera sulla propria copia dell'applicazione, o su un'istanza condivisa dove le sessioni sono indipendenti e non persistenti.

V-02 — Sessioni non persistenti. Il progresso dello studente nel simulatore non viene salvato tra le sessioni. Ogni caricamento della pagina riparte dallo stato iniziale dell'esercizio. Questa scelta è coerente con l'uso in contesto laboratoriale, dove la sessione ha una durata definita e il completamento avviene tipicamente in un'unica seduta.

V-03 — Connettività richiesta solo al caricamento. La connessione di rete è necessaria per il caricamento iniziale dell'applicazione e della configurazione. Una volta caricata, la simulazione opera interamente lato client e non richiede ulteriori comunicazioni con il server.

V-04 — Browser moderno. Si assume che gli studenti utilizzino un browser moderno (Chrome, Firefox, Edge, Safari nelle versioni correnti) con supporto completo per ES2017+, CSS Grid, e le API Web standard (`localStorage`, `Fetch`, `DOM Events`).

V-05 — Conoscenze tecniche dell'autore. Si assume che l'autore abbia familiarità con i concetti di sicurezza hardware trattati (UART, SPI, analisi firmware) e con la struttura logica di un sistema embedded. Il pannello di configurazione non fornisce spiegazioni didattiche sui protocolli, ma offre un'interfaccia strutturata per la loro configurazione.

Capitolo 2

Progettazione dell'architettura

2.1 Scelte tecnologiche

La fase di progettazione ha richiesto, anzitutto, una riflessione attenta sullo stack tecnologico da adottare. I vincoli di partenza erano chiari: la piattaforma doveva funzionare interamente nel browser, senza richiedere installazioni lato studente, e doveva essere sufficientemente flessibile da consentire agli autori di progettare esercizi eterogenei — dalla semplice ispezione visiva di un circuito stampato fino all'estrazione di firmware via protocollo SPI.

Si è optato per **Next.js 15** come framework applicativo, nella sua configurazione con App Router e output standalone. La scelta non è casuale: Next.js consente di gestire in un unico progetto sia le pagine interattive del simulatore sia le API REST necessarie alla persistenza delle configurazioni, evitando la complessità di mantenere un backend separato. La modalità standalone, in particolare, produce un bundle autocontenuto che semplifica notevolmente il deployment in ambienti containerizzati.

Per quanto riguarda la libreria di interfaccia, si è adottato **React 19** con **TypeScript**. La scelta di TypeScript è stata determinante per un progetto di questa complessità: il modello dati dell'esercizio coinvolge decine di interfacce interconnesse — pin, obiettivi, step, configurazioni di strumenti — e la tipizzazione statica ha permesso di intercettare intere classi di errori in fase di sviluppo piuttosto che a runtime, quando la diagnosi sarebbe stata sensibilmente più costosa.

Lo stile dell'interfaccia è gestito tramite **Tailwind CSS** nella versione 4, affiancato da componenti primitivi di **Radix UI** per gli elementi modali e i tooltip. Radix fornisce componenti headless — cioè privi di stile predefinito — che garantiscono accessibilità e comportamento corretto (gestione del focus, chiusura con Escape,

overlay) senza imporre vincoli estetici. Questo approccio consente di mantenere un linguaggio visivo coerente con il resto dell'applicazione.

Per la gestione dello stato si è scelto **Zustand**, una libreria di state management che si distingue per la sua leggerezza e per l'assenza di boilerplate. A differenza di Redux, Zustand non richiede reducer, action creator o middleware complessi: lo stato è definito come un semplice oggetto JavaScript e le azioni sono funzioni che lo mutano direttamente. Questa semplicità è risultata particolarmente vantaggiosa in un contesto dove coesistono quattro store indipendenti, ciascuno con responsabilità ben definite.

La scelta di non adottare un database relazionale per la persistenza è stata intenzionale. Trattandosi di un'applicazione destinata all'uso didattico in sessioni singole, si è privilegiata la semplicità: le configurazioni degli esercizi vengono salvate come file JSON su disco (lato server) e in `localStorage` (lato client). Questo approccio elimina la necessità di un DBMS, riduce le dipendenze infrastrutturali e rende il progetto immediatamente eseguibile con un semplice `npm run dev`.

2.2 Architettura generale del sistema

L'applicazione è strutturata attorno a due modalità operative distinte, ciascuna servita da una route dedicata:

- La route principale (/) ospita il **simulatore**, ovvero l'ambiente in cui lo studente interagisce con il PCB virtuale, utilizza gli strumenti di misura e risolve progressivamente gli obiettivi dell'esercizio.
- La route `/settings` ospita il **pannello di authoring**, dove il docente o l'autore dell'esercizio definisce la disposizione dei componenti sul circuito, configura i pin, imposta gli obiettivi e progetta il comportamento del terminale.

Questa separazione non è soltanto logica ma anche strutturale. Le due modalità condividono il modello dati — la struttura **Exercise** e le interfacce correlate — ma operano su store Zustand differenti e non comunicano direttamente tra loro. Il flusso di dati tra authoring e simulazione avviene attraverso la persistenza: l'autore salva la configurazione (via API o `localStorage`), e il simulatore la carica all'avvio.

Dal punto di vista del rendering, il simulatore presenta un layout a griglia con tre aree principali: un pannello laterale sinistro con gli strumenti disponibili, un'area centrale che ospita alternativamente il visualizzatore PCB o il terminale, e un pannello superiore con le istruzioni dell'obiettivo corrente e l'indicatore di progresso della flag. Il passaggio tra PCB e terminale è mutuamente esclusivo — quando

il terminale è attivo, il PCB viene nascosto e viceversa — una scelta dettata dalla volontà di mantenere l'interfaccia pulita e focalizzare l'attenzione dello studente sull'attività in corso.

Il pannello di authoring adotta una struttura analoga, con una sidebar laterale organizzata in tab tematiche (componenti, pin, terminale, strumenti) e un canvas centrale dove l'autore posiziona visivamente gli elementi sulla foto del PCB.

Le API REST, implementate come route handler di Next.js all'interno della directory `src/app/api/`, gestiscono la persistenza delle configurazioni, il caricamento dei preset, e l'upload di immagini e file firmware. Operano tutte su file system locale, coerentemente con la scelta di evitare dipendenze esterne.

2.3 Modello dei dati

Il modello dati costituisce il cuore progettuale della piattaforma. L'interfaccia principale, **Exercise**, rappresenta un esercizio completo e contiene al suo interno tutte le informazioni necessarie sia al rendering nel simulatore sia alla validazione del progresso dello studente.

2.3.1 Struttura dell'esercizio

Un esercizio è organizzato in **step** sequenziali, ciascuno dei quali contiene uno o più **obiettivi**. Ogni step rappresenta una fase dell'esercitazione — ad esempio “Identificare il chip di memoria flash” o “Estrarre il firmware” — e definisce una flag attesa (`expectedFlag`) che lo studente deve ricostruire completando gli obiettivi. Ogni obiettivo, a sua volta, contribuisce con un frammento di flag (`flagPart`); la flag finale dell'esercizio è la concatenazione ordinata di tutti i frammenti degli obiettivi completati.

Ogni obiettivo (**Objective**) è tipizzato secondo una delle cinque categorie supportate:

- **component**: richiede di identificare e cliccare un componente specifico sulla foto del PCB;
- **pin**: richiede di collegare le sonde del multimetro a pin specifici, verificando che i valori misurati soddisfino condizioni predefinite;
- **uart**: richiede di stabilire una connessione UART corretta tra l'adattatore e i pin del circuito;

- **terminal**: richiede di interagire con il terminale simulato per scoprire flag nascoste tramite l'esecuzione di comandi;
- **firmware-dump**: richiede di collegare le sonde SPI ai pin corrispondenti ed eseguire il dump del firmware.

Questa tassonomia copre le attività più comuni nell'analisi hardware reale, dalla ricognizione visiva all'estrazione di dati, offrendo un percorso didattico graduato.

2.3.2 Il sistema unificato dei pin

Un aspetto progettuale significativo riguarda la gestione dei pin. Il circuito simulato può contenere tre tipologie di pin, ciascuna con semantica e ruoli diversi:

- **Pin di misura** (`MeasurementPin`): rappresentano punti di test generici sul PCB, caratterizzati da un valore di tensione (`valueV`) e un valore di resistenza (`valueOhm`). Sono i pin con cui lo studente interagisce tramite il multimetro.
- **Pin UART** (`UartPin`): rappresentano i pin del bus seriale UART, ciascuno con un ruolo specifico (`tx`, `rx`, `gnd`, `vcc`). Il collegamento corretto richiede che lo studente rispetti la corrispondenza incrociata tra i pin dell'adattatore e quelli del circuito (il TX dell'adattatore va collegato all'RX del dispositivo e viceversa).
- **Pin SPI** (`FirmwareDumpPin`): rappresentano i pin del bus SPI utilizzato per la comunicazione con memorie flash, ciascuno con un ruolo tra `vcc`, `gnd`, `cs` (chip select), `clk` (clock), `mosi` (master out, slave in) e `miso` (master in, slave out).

Nonostante le differenze semantiche, tutti e tre i tipi condividono lo stesso sistema di coordinate e lo stesso meccanismo di overlay. Questa uniformità è resa possibile da un insieme di funzioni helper — `getAllPins()`, `getPinCoords()`, `getPinValues()` — che astraggono le differenze tra i tipi e restituiscono una vista unificata. Il vantaggio è duplice: il codice di rendering può trattare i pin in modo omogeneo, e il meccanismo di snap (aggancio automatico delle sonde ai pin vicini) funziona con un unico algoritmo indipendentemente dal tipo di pin.

2.3.3 Il sistema di coordinate

Tutte le posizioni degli elementi interattivi — componenti, pin, overlay — sono espresse in **coordinate normalizzate**, ovvero come percentuali della larghezza e

dell'altezza del contenitore. Un pin posizionato a [50, 30, 2, 2] si trova al centro orizzontale del PCB, al 30% dall'alto, con dimensione pari al 2% del contenitore in entrambe le direzioni.

Questa scelta rende il layout intrinsecamente responsivo: al variare delle dimensioni della finestra, tutti gli elementi si riposizionano automaticamente mantenendo le proporzioni relative. La conversione da coordinate normalizzate a pixel avviene nel componente `PCBViewer`, che tiene traccia delle dimensioni reali del contenitore tramite `getBoundingClientRect()` e ricalcola le posizioni ad ogni ridimensionamento.

2.4 Gestione dello stato applicativo

La gestione dello stato rappresenta uno degli aspetti più delicati dell'architettura, dato il numero di interazioni simultanee che il simulatore deve gestire: posizione delle sonde, connessioni ai pin, stato del multimetro, progresso dell'esercizio, stato del terminale, e così via.

2.4.1 Architettura a store indipendenti

Si è adottata un'architettura a **quattro store Zustand indipendenti**, ciascuno con un ambito di responsabilità ben circoscritto:

Store	Responsabilità
<code>exerciseStore</code>	Stato runtime del simulatore: progresso step/obiettivi, selezione strumenti, connessioni sonde, flag scoperte, stato del terminale
<code>settingsStore</code>	Stato della modalità authoring: trasformazioni canvas, componenti e pin in fase di editing, strumenti personalizzati, configurazione degli strumenti
<code>terminalSettingsStore</code>	Editor della configurazione terminale: componenti terminale, tab, comandi, filesystem, sequenze di boot, flag
<code>presetStore</code>	Gestione dei preset: salvataggio e caricamento di bundle che combinano configurazione esercizio e terminale

La scelta di mantenere gli store separati e privi di un reducer comune è stata ponderata. In un sistema come Redux, la presenza di un root reducer centralizzato facilita la coordinazione ma introduce accoppiamento: ogni azione deve attraversare

un singolo punto di ingresso, e la modifica di uno store può avere effetti collaterali su altri. Con Zustand, ogni store è autocontenuto e può essere consumato indipendentemente dai componenti che ne hanno bisogno, riducendo i re-render non necessari.

2.4.2 Coordinazione tra store

L'assenza di un reducer comune non implica l'assenza di coordinazione. Quando è necessaria una comunicazione tra store — ad esempio, per tracciare se la configurazione corrente è stata modificata rispetto al preset caricato — si utilizzano le **subscription** di Zustand.

Il caso più rilevante riguarda il **presetStore**, che si sottoscrive ai cambiamenti di **settingsStore** e **terminalSettingsStore** per determinare lo stato “dirty” (modificato ma non salvato). La sottoscrizione confronta per referenza i campi rilevanti — componenti, step, pin, comandi, filesystem — e ignora i campi puramente di interfaccia come lo zoom del canvas o l'elemento attualmente selezionato. Un meccanismo di guardia impedisce che il caricamento di un preset venga erroneamente interpretato come una modifica: durante il caricamento, un flag **isLoading** disabilita temporaneamente il rilevamento dei cambiamenti.

Questa coordinazione è esplicita e tracciabile, a differenza di pattern più opachi come gli event bus globali. Ogni sottoscrizione è dichiarata in modo visibile nel codice del presetStore, rendendo immediatamente comprensibile quali cambiamenti vengano osservati e quale effetto producano.

2.4.3 Persistenza dello stato

La persistenza segue strategie diverse a seconda dello store:

- **settingsStore** utilizza una persistenza manuale: l'autore salva esplicitamente la configurazione tramite un'azione che serializza lo stato in JSON e lo scrive in **localStorage** (chiave **pcb-ctf-exercise-config**), oltre a inviarlo al server via API per la scrittura su file.
- **terminalSettingsStore** adotta una strategia ibrida. Lo stato di draft dell'editor viene persistito automaticamente in **localStorage** tramite il middleware **persist** di Zustand (chiave **pcb-ctf-terminal-settings-draft**), garantendo che il lavoro in corso sopravviva al refresh della pagina. La configurazione terminale “applicata” — quella effettivamente usata dal simulatore — viene scritta in una chiave separata (**pcb-ctf-terminal-config**).

- **exerciseStore** non prevede persistenza: il suo stato è interamente transiente. L'esercizio viene caricato ad ogni avvio dalla configurazione salvata, e il progresso dello studente si resetta con il refresh della pagina. Questa scelta riflette la natura “da laboratorio” della piattaforma, dove ogni sessione è indipendente.
- **presetStore** persiste unicamente l'identificativo del preset attivo (chiave `pcb-ctf-active-p`) per ripristinare la selezione nel menu a tendina dopo un ricaricamento della pagina.

2.4.4 Migrazione dei dati

Un aspetto che merita menzione è la gestione della retrocompatibilità. Il sistema dei terminali è stato inizialmente progettato con una singola configurazione monolitica; in una fase successiva, si è evoluto verso un'architettura modulare a componenti multipli. La funzione di merge del middleware **persist** gestisce trasparentemente la migrazione: se rileva uno stato nel formato precedente (senza l'array `terminalComponents`), lo converte automaticamente nel nuovo formato, avvolgendo i dati esistenti in un componente predefinito. Questo approccio ha evitato la necessità di script di migrazione esterni e ha garantito la continuità per chi aveva già configurazioni salvate.

2.5 Progettazione del sistema terminale

Il terminale simulato è, senza dubbio, il sottosistema più complesso dell'intera piattaforma. A differenza degli strumenti di misura — che operano con interazioni puntuali (collegare una sonda, leggere un valore) — il terminale deve simulare un ambiente interattivo completo, con filesystem, comandi, vincoli di accesso e una sequenza di boot che replica il comportamento di un dispositivo reale.

2.5.1 Principi di progettazione

La decisione fondamentale, presa nelle fasi iniziali del progetto, è stata quella di implementare un terminale **interamente lato client**, senza alcuna esecuzione reale di comandi. Ogni interazione è gestita da un motore di esecuzione configurabile che interpreta le definizioni dei comandi contenute nella configurazione JSON.

Questa scelta comporta limitazioni evidenti — lo studente non può eseguire comandi arbitrari, ma solo quelli previsti dall'autore — ma offre vantaggi decisivi per il contesto d'uso:

1. **Sicurezza:** nessun rischio di code injection o di accesso non autorizzato a risorse del server;
2. **Determinismo:** il comportamento del terminale è completamente prevedibile e riproducibile;
3. **Portabilità:** l'applicazione non dipende da un sistema operativo specifico per l'esecuzione dei comandi;
4. **Controllo didattico:** l'autore ha il pieno controllo su quali comandi sono disponibili in ogni fase dell'esercizio.

2.5.2 Architettura modulare

Il sistema terminale adotta un'architettura a **componenti modulari**: un esercizio può contenere più terminali indipendenti, ciascuno con il proprio set di tab, comandi, filesystem e flag. Un terminale potrebbe simulare la console UART di un router, un altro il terminale di una macchina Kali Linux utilizzata per l'analisi.

Ogni componente terminale (`TerminalComponent`) incapsula:

- un insieme di **tab**, ciascuno con il proprio contesto di esecuzione;
- un **filesystem simulato** con directory e file;
- una **sequenza di boot** opzionale con stadi animati;
- un **sistema di flag** con parti da scoprire progressivamente;
- un set di **comandi** con vincoli, output e side effect.

2.5.3 Pipeline di esecuzione dei comandi

L'esecuzione di un comando nel terminale attraversa una pipeline a quattro stadi:

1. **Parsing:** l'input dell'utente viene suddiviso in nome del comando e argomenti.

2. **Validazione dei vincoli (constraints):** prima dell'esecuzione, il sistema verifica un insieme di condizioni che il comando potrebbe imporre:

- vincolo di **percorso**: il comando è disponibile solo in determinate directory;
- vincolo di **permessi**: richiede un utente specifico (es. root);
- vincolo di **prerequisiti**: richiede che altri comandi siano già stati eseguiti o che specifiche flag siano state scoperte;

- vincolo di **argomenti**: numero e formato degli argomenti;
- vincolo di **stato**: disponibilità limitata a specifici stadi di boot.

Se un vincolo non è soddisfatto, il comando non viene eseguito e viene restituito un messaggio di errore contestuale.

3. Generazione dell'output: se i vincoli sono soddisfatti, il sistema produce l'output secondo il tipo configurato per il comando. Sono supportati sei tipi di output, descritti in dettaglio nel capitolo successivo.

4. Applicazione dei side effect: dopo l'esecuzione, il comando può produrre effetti collaterali — sblocco di flag, cambiamento dello stadio di boot, modifica del percorso corrente. Questi effetti vengono restituiti al componente React, che li propaga allo store e aggiorna di conseguenza l'interfaccia.

2.5.4 Il filesystem simulato

Il filesystem è implementato come una struttura a due livelli: un record di **directory** che mappa ogni percorso alla lista dei suoi figli diretti, e un record di **file** che mappa ogni percorso completo ai metadati del file (contenuto, permessi, proprietario, dimensione).

La configurazione supporta una notazione compatta ad albero (**tree**) che viene automaticamente “appiattita” dal loader in fase di caricamento. L'autore può così definire la struttura del filesystem in modo leggibile:

```
{
  "tree": {
    "bin": { "busybox": "...", "sh": "..." },
    "etc": { "passwd": "root:x:0:0:..." },
    "var": { "log": { "syslog": "..." } }
  }
}
```

Il loader converte questa struttura nidificata nel formato piatto atteso dal motore di esecuzione, calcolando automaticamente le entry di ogni directory.

2.5.5 La sequenza di boot

Per gli esercizi che simulano l'accesso a dispositivi embedded, il terminale supporta una sequenza di boot animata. La sequenza è definita come un elenco di **stadi**, ciascuno con un identificativo, un set di righe da visualizzare, una durata di animazione e un eventuale stadio successivo.

L'animazione riproduce il flusso di un boot reale: le righe di log appaiono progressivamente, con un intervallo calcolato dividendo la durata totale dello stadio per il numero di righe. Alcuni stadi prevedono interazione — ad esempio, uno stadio di tipo “login” attende che lo studente inserisca le credenziali corrette prima di procedere — replicando il comportamento di un accesso seriale via UART a un dispositivo di rete.

2.5.6 Il sistema di flag

Il terminale implementa un meccanismo di **flag progressive**: la flag completa è suddivisa in parti (**FlagPart**), ciascuna associata a un identificativo e a un frammento di testo. Lo studente scopre le parti esplorando il filesystem e eseguendo i comandi giusti; l'interfaccia mostra in tempo reale il progresso, visualizzando i frammenti già scoperti e sostituendo con punti interrogativi quelli ancora da trovare.

Lo sblocco di una flag avviene tramite i side effect dei comandi: quando un comando prevede **unlockFlags** tra i suoi effetti collaterali, il sistema aggiunge gli identificativi corrispondenti all'elenco delle flag scoperte, a patto che non siano già presenti. Lo sblocco può anche essere condizionale — ad esempio, subordinato al fatto che lo studente si trovi in una directory specifica o abbia già scoperto un'altra flag.

Quando tutte le parti sono state scoperte, il sistema notifica l'exerciseStore, che a sua volta verifica se l'obiettivo terminale corrente è da considerarsi completato.

2.6 Flusso dati tra authoring e simulazione

Il flusso di dati tra le due modalità operative della piattaforma è unidirezionale e mediato dalla persistenza. Non esiste comunicazione diretta tra gli store della modalità settings e quelli della modalità simulazione; il contratto tra le due parti è rappresentato dalle interfacce TypeScript condivise.

Il flusso si articola in tre fasi:

Fase 1 — Authoring: il docente configura l'esercizio attraverso il pannello settings. Le modifiche vengono accumulate negli store **settingsStore** e **terminalSettingsStore** come oggetti “draft” — versioni editabili delle entità finali. Quando l'autore è soddisfatto, invoca l'azione di salvataggio, che serializza lo stato in formato **Exercise** e **TerminalConfig**, lo scrive in **localStorage** e, tramite le API REST, lo persiste su file.

Fase 2 — Persistenza: le configurazioni vengono salvate come file JSON nella directory `src/data/`: `exercise.override.json` per la configurazione dell'esercizio,

e una struttura analoga per il terminale. I preset — bundle che combinano entrambe le configurazioni — vengono salvati nella sottodirectory `presets/`, con un file indice che ne tiene traccia.

Fase 3 — Simulazione: all'avvio del simulatore, l'hook `useExerciseConfig()` carica la configurazione dell'esercizio dalla API (`/api/config/load`), che a sua volta legge il file JSON dal disco. L'hook `useTerminalConfig()` opera in modo analogo per la configurazione del terminale, con la possibilità di risolvere uno specifico componente terminale per identificativo.

Una caratteristica utile del sistema è la **sincronizzazione in tempo reale**: l'hook `useTerminalConfig()` si sottoscrive agli eventi `storage` del browser e a un evento personalizzato `terminal-config-updated`. Questo significa che, se l'autore modifica la configurazione del terminale nella tab settings e la applica, il simulatore aperto in un'altra tab del browser riceve automaticamente la nuova configurazione senza necessità di ricaricare la pagina — una funzionalità particolarmente comoda durante lo sviluppo iterativo di un esercizio.

2.6.1 Il sistema di preset

I preset rappresentano il meccanismo principale per il riutilizzo e la distribuzione degli esercizi. Un preset è un bundle immutabile al momento della creazione che contiene sia la configurazione dell'esercizio sia quella del terminale, corredato di metadati (nome, descrizione, data di creazione).

La gestione dei preset è affidata a un set di API REST dedicate che operano su file JSON nella directory `src/data/presets/`. Un file indice (`index.json`) mantiene l'elenco dei preset disponibili con i relativi metadati, mentre ciascun preset è salvato in un file separato identificato da uno slug generato dal nome.

Un aspetto degno di nota è la gestione delle immagini del PCB: al salvataggio di un preset, l'immagine corrente viene copiata in un percorso dedicato al preset, garantendone la sopravvivenza anche se l'autore carica successivamente un'immagine diversa. Questa strategia assicura che ogni preset sia autocontenuto e riproducibile.

Capitolo 3

Implementazione

3.1 Il visualizzatore PCB interattivo

Il componente centrale del simulatore è il **PCBViewer**, responsabile del rendering della fotografia del circuito stampato e di tutti gli elementi interattivi sovrapposti: pin, sonde, fili, componenti cliccabili e lente di ingrandimento. La complessità di questo componente deriva dalla necessità di gestire simultaneamente molteplici livelli di interazione, ciascuno con il proprio comportamento di input e la propria logica di rendering.

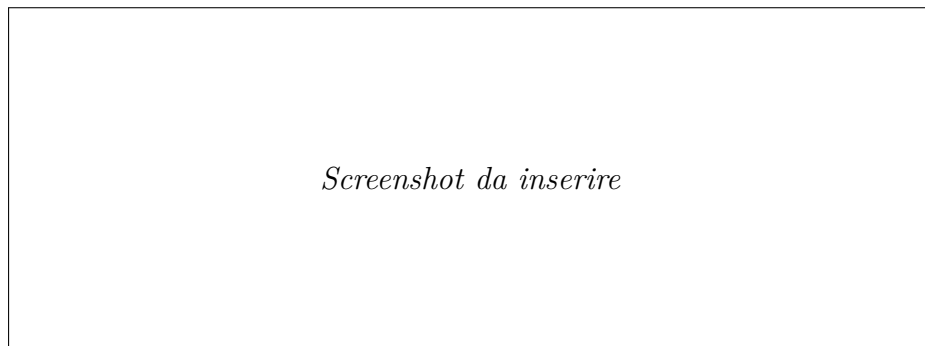


Figura 3.1: Interfaccia principale del simulatore: a sinistra la sidebar con gli strumenti, al centro il PCBViewer con overlay dei componenti e pin, in alto le istruzioni dell'obiettivo corrente.

3.1.1 Sistema di overlay e conversione delle coordinate

Il PCBViewer opera come un contenitore di coordinate. L'immagine del PCB occupa l'intera area disponibile, e sopra di essa vengono posizionati — tramite posizionamento CSS assoluto — gli overlay dei componenti, i pin, gli strumenti trascinabili e i fili di collegamento.

La conversione tra coordinate normalizzate (percentuali) e pixel avviene ad ogni render e ad ogni ridimensionamento della finestra. Il componente mantiene un riferimento alle dimensioni reali del contenitore (`containerDims`), ottenute tramite `getBoundingClientRect()`, e utilizza queste dimensioni come fattore di scala. Un pin definito alla posizione `[25, 60]` nella configurazione verrà renderizzato al 25% della larghezza e al 60% dell'altezza del contenitore, indipendentemente dalla risoluzione dello schermo.

I pin sono renderizzati come piccoli indicatori circolari semi-trasparenti, la cui visibilità dipende dallo strumento attivo: i pin di misura appaiono solo con il multimetro, i pin UART solo con le sonde UART, i pin SPI solo con lo strumento di firmware dump. Questa stratificazione contestuale mantiene l'interfaccia pulita e guida lo studente verso le interazioni pertinenti.

3.1.2 Meccanismo di snap

L'interazione sonda-pin utilizza un meccanismo di **snap** (aggancio automatico) che scatta quando una sonda trascinata si avvicina sufficientemente a un pin compatibile. Il raggio di aggancio è calcolato dinamicamente come il massimo tra 15 pixel e metà della larghezza del pin, adattandosi così a diverse risoluzioni e dimensioni di visualizzazione.

Quando il cursore entra nel raggio di aggancio di un pin, lo store viene aggiornato con l'identificativo del pin candidato (`snapTarget`), e il pin corrispondente viene evidenziato visivamente con un'animazione pulsante. Se l'utente rilascia la sonda in questo stato, la connessione viene stabilita e il filo assume un tracciato fisso. Il filtraggio dei pin per tipo — necessario perché le sonde del multimetro non devono agganciarsi ai pin UART e viceversa — viene gestito a monte, nel calcolo del pin più vicino.

3.1.3 Rendering dei fili

I fili che collegano le sonde ai pin sono renderizzati come **curve di Bézier quadratiche** all'interno di un elemento SVG sovrapposto all'intera area del PCB. La scelta delle curve di Bézier, piuttosto che segmenti rettilinei, produce un aspetto visivamente più naturale, evocativo di un cavo fisico che segue una traiettoria morbida.

I fili seguono una distinzione visiva tra stato attivo e connesso: durante il trasciamento, il filo dalla sonda al cursore è tratteggiato e semi-trasparente, suggerendo una connessione potenziale; una volta stabilita la connessione, il filo diventa con-

tinuo e completamente opaco. La codifica cromatica è specifica per ogni tipo di strumento: il multimetro usa rosso e nero per le due sonde, le connessioni UART seguono i colori convenzionali del bus seriale (verde per TX, giallo per RX, grigio per GND), e i fili SPI utilizzano una palette a sei colori distinti per i ruoli VCC, GND, CS, CLK, MOSI e MISO.

3.1.4 La lente di ingrandimento

Lo strumento magnifier implementa una lente di ingrandimento circolare che consente allo studente di esaminare da vicino i dettagli del PCB — la serigrafia dei componenti, le sigle dei chip, le piste del circuito. L’implementazione sfrutta la proprietà CSS `background-image` con `background-size` scalata per il fattore di zoom e `background-position` calcolata in funzione della posizione del cursore, ottenendo così un effetto lente in tempo reale senza duplicare il DOM.

La lente può essere ancorata in una posizione fissa tramite un click, liberando il cursore per altre interazioni — una funzionalità utile quando lo studente vuole leggere un’etichetta mentre consulta le istruzioni. Il raggio della lente e il livello di zoom sono configurabili dall’autore dell’esercizio, permettendo di calibrare lo strumento in base alla risoluzione dell’immagine del PCB.

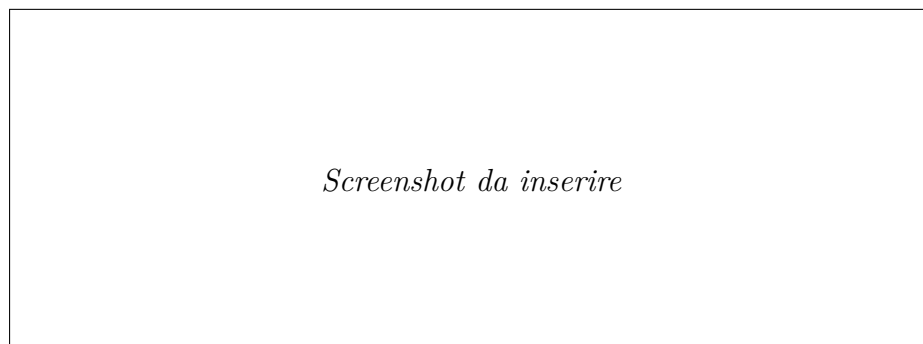


Figura 3.2: Lente di ingrandimento attiva sulla fotografia del PCB, che mostra i dettagli della serigrafia di un chip.

Un aspetto non banale dell’implementazione riguarda il rendering del contenuto “sotto” la lente. Non è sufficiente zoomare l’immagine di sfondo: anche i pin, i fili e gli overlay dei componenti devono apparire ingranditi coerentemente all’interno dell’area della lente. Questo viene gestito da un componente dedicato (`LensContentLayer`) che ricalcola le posizioni di tutti gli elementi visibili applicando la stessa trasformazione di scala e offset della lente.

3.2 Strumenti di analisi hardware

3.2.1 Il multimetro

Il multimetro digitale è il primo strumento che lo studente incontra tipicamente in un esercizio. La sua implementazione replica il comportamento di un multimetro reale, con due sonde (rossa e nera), un display a cristalli liquidi e un selettore di modalità (tensione o resistenza).

Il componente è trascinabile — l'utente lo posiziona sulla scrivania virtuale afferrando l'header — e presenta due “manici” per le sonde, anch'essi trascinabili. La lettura sul display segue regole che riflettono il funzionamento reale dello strumento:

- In **modalità tensione con una sola sonda** collegata, il display mostra la tensione assoluta del pin rispetto a un riferimento implicito di massa.
- In **modalità tensione con entrambe le sonde**, il display mostra la differenza di potenziale $V(\text{rossa}) - V(\text{nera})$, con il segno che indica la polarità.
- In **modalità resistenza**, entrambe le sonde devono essere collegate; il display mostra la somma delle resistenze dei pin connessi, con la visualizzazione “OL” (overload) oltre una soglia configurabile.

Per conferire realismo alla simulazione, il valore visualizzato è affetto da un **rumore digitale** di ± 0.04 unità, aggiornato ogni 400 millisecondi tramite un intervallo periodico. Questo piccolo accorgimento riproduce l'instabilità tipica delle ultime cifre di un multimetro reale e contribuisce a rendere l'esperienza più credibile.

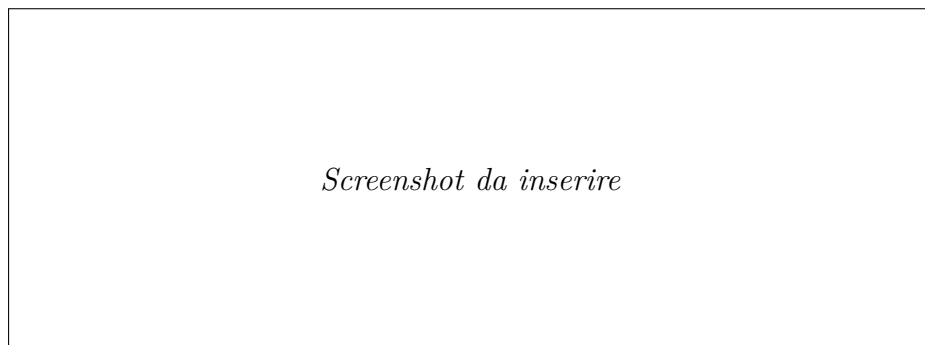


Figura 3.3: Multimetro digitale con le due sonde collegate a pin del PCB. Il display mostra la lettura di tensione con il rumore digitale simulato.

3.2.2 L'adattatore UART

L'adattatore UART simula un convertitore USB-seriale, lo strumento che nella pratica reale viene utilizzato per stabilire una comunicazione seriale con il dispositivo

target. L'interfaccia mostra un corpo rettangolare con quattro slot per i pin dell'adattatore: TX, RX, GND e VCC (quest'ultimo disabilitato nell'implementazione corrente).

L'interazione richiede che lo studente colleghi ciascun pin dell'adattatore al corrispondente pin sul PCB, rispettando la logica di connessione incrociata propria del protocollo UART: il pin TX dell'adattatore deve essere collegato al pin RX del dispositivo, e il pin RX dell'adattatore al pin TX del dispositivo. La massa (GND) va collegata a GND.

La **validazione** del collegamento avviene in fase di hook: quando lo studente rilascia una sonda su un pin, il sistema confronta il ruolo della sonda con il ruolo del pin di destinazione. Se il ruolo non corrisponde secondo la mappatura prevista (TX→RX, RX→TX, GND→GND), la connessione viene rifiutata e la sonda torna alla posizione di partenza. Questo meccanismo offre un feedback immediato senza rivelare esplicitamente la soluzione, incoraggiando lo studente a ragionare sulla logica del protocollo.

Una volta completate tutte le connessioni richieste, l'adattatore può **attivare automaticamente un terminale**, simulando l'apertura di una sessione seriale sul dispositivo.

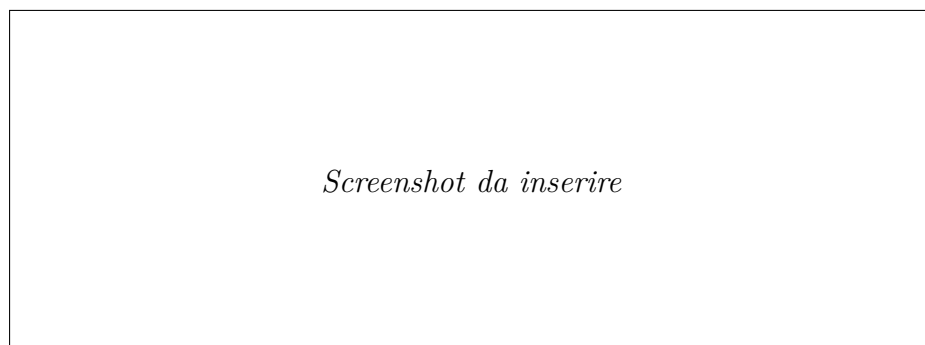


Figura 3.4: Adattatore UART con le sonde TX, RX e GND correttamente collegate ai pin del PCB secondo la logica di crossover. L'identificativo del componente terminale da lanciare è configurabile dall'autore tramite il campo `terminalComponentId` nella configurazione dell'adattatore UART.

3.2.3 Il firmware dumper SPI

Lo strumento di firmware dump replica il processo di estrazione del contenuto di una memoria flash SPI — un'operazione comune nell'analisi hardware reale, dove si utilizza un programmatore come il Bus Pirate o il CH341A per leggere il chip di memoria.

L'interfaccia presenta sei sonde colorate, una per ciascun segnale del bus SPI (VCC, GND, CS, CLK, MOSI, MISO), disposte in una griglia etichettata. Lo studente deve trascinare ciascuna sonda sul pin corrispondente del PCB, rispettando la corrispondenza di ruolo: la sonda CLK va sul pin CLK, la sonda MISO sul pin MISO, e così via.

La validazione supporta due modalità, a seconda della configurazione dell'esercizio. La modalità predefinita è **role-based**: ogni sonda ha un ruolo SPI e il pin di destinazione deve avere lo stesso ruolo — se lo studente tenta di collegare la sonda CS al pin CLK, la connessione viene rifiutata. In alternativa, l'autore può definire un mapping esatto tramite il campo `requiredConnections`, specificando per ogni sonda l'identificativo preciso del pin a cui deve essere collegata. Questa seconda modalità è utile quando il PCB presenta pin che non corrispondono ai ruoli SPI standard.

Il flusso operativo dello strumento prevede diverse fasi. Una volta completati tutti i collegamenti, il comportamento dipende dalla configurazione: se il firmware dump è associato a un componente terminale (tramite `terminalComponentId`), il sistema lancia automaticamente il terminale *prima* di abilitare il dump, richiedendo allo studente di completare i comandi terminale come prerequisito. L'interfaccia mostra un messaggio esplicito (“Complete terminal commands first”) e il pulsante di dump resta disabilitato fino al completamento della sfida terminale. Questo flusso replica scenari realistici in cui l'analista deve prima interagire con il dispositivo via terminale per identificare il chip di memoria, e solo successivamente procedere con l'estrazione.

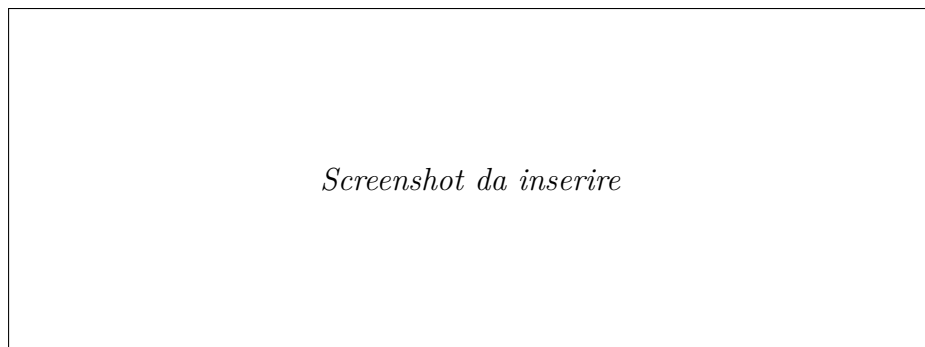


Figura 3.5: Firmware dumper SPI con le sonde collegate e la barra di progresso del dump in corso.

Quando il dump viene avviato, una **barra di progresso** animata simula il trasferimento dei dati. La durata è configurabile (campo `dumpDurationSec`) e al termine viene mostrato un riquadro di conferma con il pulsante per scaricare il file firmware — un file binario reale, caricato dall'autore tramite l'apposita API

di upload. Il download e il proseguimento dell'esercizio sono gestiti dall'azione `dismissFirmwareDumpDownload()`, che separa deliberatamente il completamento del dump dalla progressione dell'obiettivo, permettendo allo studente di visualizzare e scaricare il file prima che il flusso avanzi.

3.3 Il terminale simulato

L'implementazione del terminale traduce in codice l'architettura descritta nel capitolo precedente. Il sistema si compone di tre moduli principali — il loader della configurazione, l'esecutore dei comandi e gli handler dei comandi built-in — coordinati dal componente `React Terminal.tsx`.

3.3.1 Il loader della configurazione

La classe `TerminalConfigLoader` è responsabile della preparazione della configurazione terminale per l'uso a runtime. Il processo di caricamento si articola in quattro fasi sequenziali:

1. **Appiattimento dell'albero:** la struttura `tree` nidificata del filesystem viene convertita nel formato piatto a due record (directory e file) utilizzato dal motore di esecuzione. Un albero `{ "etc": { "passwd": "root:..." } }` diventa le entry `/etc` nelle directory e `/etc/passwd` nei file.
2. **Normalizzazione dei file:** le stringhe abbreviate vengono espanse in oggetti `FileNode` completi, con permessi predefiniti (`-rw-r-r-`), proprietario (`root`), e dimensione calcolata dal contenuto.
3. **Calcolo delle entry di directory:** per ogni directory, il loader scansiona tutte le chiavi dei record directory e file per determinarne i figli diretti, eliminando la necessità per l'autore di specificarli manualmente.
4. **Costruzione della mappa dei comandi:** i comandi globali vengono uniti a quelli specifici di ogni tab, applicando i vincoli predefiniti del tab (`defaultConstraints`) a ogni comando che non definisce i propri. Vengono inoltre registrati gli alias, che permettono di definire nomi alternativi per lo stesso comando.

Il risultato è un oggetto cacheable che espone metodi per accedere a comandi, filesystem, sequenze di boot e flag per qualunque tab.

3.3.2 L'esecutore dei comandi

La classe `TerminalCommandExecutor` implementa la pipeline di esecuzione in quattro stadi descritta nel capitolo precedente. L'aspetto più interessante riguarda la molteplicità dei tipi di output supportati, che conferiscono al terminale simulato una flessibilità notevole.

Output statico: il tipo più semplice, restituisce un array di stringhe predefinito. È adatto a comandi il cui output non dipende dal contesto — ad esempio un banner di benvenuto o un messaggio di aiuto.

Output condizionale: il sistema valuta una catena di condizioni e restituisce l'output del primo ramo soddisfatto. Le condizioni possono verificare il valore di un argomento, il percorso corrente, l'esistenza di un file, lo stato di una flag, o il valore di una variabile di stato. Questo tipo permette di costruire comandi che si comportano diversamente in base al contesto — ad esempio, un comando `strings` che restituisce output diversi a seconda del file analizzato.

Output template: utilizza stringhe con segnaposto `{{variabile}}` che vengono sostituiti a runtime. Le variabili possono essere statiche (valori fissi), calcolate (da funzioni registrate), casuali (valore in un intervallo) o derivate dallo stato (variabili di ambiente). Questo tipo è utile per output parametrici come prompt personalizzati o messaggi che includono informazioni dinamiche.

Output lookup: confronta un argomento del comando con una tabella di associazioni (per uguaglianza, contenimento o regex) e restituisce l'output corrispondente, o un output predefinito se nessuna chiave corrisponde. Questo tipo è particolarmente efficace per simulare comandi come `file` o `hexdump`, il cui output dipende interamente dall'argomento fornito.

Output dinamico e script: invocano funzioni JavaScript personalizzate registrate nel contesto dell'esecutore. La differenza tra i due è sottile: `dynamic` passa argomenti aggiuntivi alla funzione, mentre `script` la invoca senza parametri. Queste modalità rappresentano la valvola di sfogo del sistema, consentendo all'autore di implementare comportamenti che non rientrano negli altri tipi.

3.3.3 I comandi built-in

Il terminale include implementazioni pure JavaScript di sette comandi standard di un sistema Unix: `ls`, `cd`, `cat`, `pwd`, `grep`, `find` e `clear`. Questi comandi operano sul filesystem simulato e producono output coerente con le rispettive controparti reali.

L'implementazione di `ls`, ad esempio, supporta i flag `-l` (formato lungo, con permessi, proprietario e dimensione) e `-a` (inclusione dei file nascosti). L'output in

formato lungo replica la formattazione caratteristica di un sistema Linux:

```
drwxr-xr-x  1 root root  4096 Jan 15 10:30 bin
-rw-r--r--  1 root root   234 Jan 15 10:30 passwd
```

Il comando **grep** implementa la ricerca con espressioni regolari, con supporto per la ricerca case-insensitive e l'indicazione del numero di riga. Il comando **find** effettua una discesa ricorsiva nelle directory a partire dal percorso specificato, con supporto per il filtraggio per nome (con wildcard) e per tipo (file o directory).

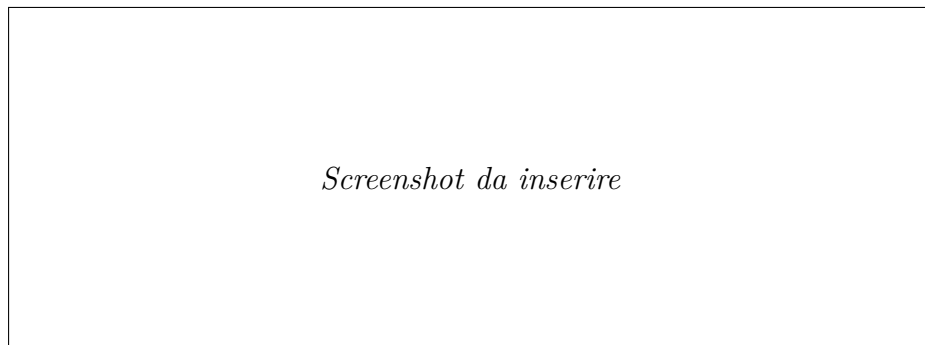


Figura 3.6: Terminale simulato durante la sequenza di boot U-Boot, con il prompt interattivo per l'accesso alla shell del bootloader.

Il comando **clear** merita una menzione per il suo meccanismo: restituisce un marker speciale (`__CLEAR__`) che il componente React intercetta e interpreta come istruzione di svuotare la cronologia visualizzata, anziché come output da visualizzare.

Tutti i comandi built-in vengono registrati nell'esecutore tramite la funzione `registerBuiltinHandlers()`, che associa ciascun nome di comando alla funzione handler corrispondente.

3.3.4 La sequenza di boot

L'implementazione della sequenza di boot è interamente gestita dal componente React `Terminal.tsx` attraverso un effetto che si attiva ad ogni cambio di tab o di stadio.

Per ogni stadio della sequenza, il componente calcola l'intervallo temporale tra le righe dividendo la durata totale per il numero di righe da visualizzare, e utilizza un `setInterval` per aggiungerle progressivamente alla cronologia. Al termine dell'animazione, se lo stadio prevede una transizione automatica (campo `nextStage`), viene programmato il passaggio allo stadio successivo con un ritardo di 500 millisecondi.

Alcuni stadi sono gestiti in modo speciale con logica hardcoded nel componente:

- Lo stadio `uboot_wait` attende la pressione del tasto Invio per procedere, simulando il classico prompt “Hit any key to stop autoboot” dei bootloader U-Boot.
- Lo stadio `login` attende l’inserimento di un nome utente valido.
- Lo stadio `password` verifica le credenziali inserite e, in caso di successo, sblocca automaticamente una flag e procede allo stadio `shell`.

Questa gestione mista — parte configurabile, parte hardcoded — riflette un compromesso pragmatico: gli stadi speciali richiedono una logica di interazione che sarebbe eccessivamente complessa da esprimere in formato dichiarativo, mentre gli stadi generici (animazione di righe di log, transizioni automatiche) sono perfettamente serviti dalla configurazione JSON.

3.3.5 Persistenza dello stato del terminale

Un problema ricorrente nelle single-page application riguarda la perdita di stato quando un componente viene smontato e rimontato. Nel caso del terminale, questo avviene ogni volta che lo studente passa dal terminale al PCB e viceversa: React smonta il componente `Terminal` e ne perde lo stato interno.

La soluzione adottata utilizza **variabili a livello di modulo** — dichiarate al di fuori del componente React, nello scope del file — per conservare la cronologia, il percorso corrente, la cronologia dei comandi e lo stadio di boot di ciascun tab. Quando il componente viene rimontato, inizializza il proprio stato da queste variabili persistenti, ripristinando esattamente la situazione precedente.

Questa tecnica, per quanto poco ortodossa rispetto al pattern convenzionale di gestione dello stato tramite store, è particolarmente efficace in questo contesto: il dato è specifico del componente e non ha bisogno di essere condiviso con altri, ma deve sopravvivere ai cicli di mount/unmount.

3.4 Il sistema di strumenti personalizzati

Accanto agli strumenti built-in, la piattaforma offre un sistema di **strumenti personalizzati** (custom tools) che consente agli autori di definire strumenti di misura arbitrari, con le proprie sonde, modalità di output e comportamento.

3.4.1 Modello dati

Ogni strumento personalizzato è definito da un insieme di proprietà:

- un **nome** e una **descrizione** opzionale;
- un'immagine opzionale (caricata tramite l'API di upload);
- un array di **sonde** (`ToolProbe`), ciascuna con etichetta, ruolo, colore e tipo di connettività;
- un **tipo di output** tra `none` (nessuna lettura), `numeric` (display numerico), `leds` (indicatore LED), `connection-status` (stato della connessione) e `firmware-dump` (dump del firmware con barra di progresso);
- un array di **modalità** (`ToolMode`), ciascuna con nome, abbreviazione e unità di misura.

Il tipo di connettività della sonda (`connectivity`) determina a quali pin può agganciarsi: `measurement` per i pin di misura, `uart` per i pin UART, `all` per qualsiasi tipo.

3.4.2 Rendering nel simulatore

Il componente `CustomToolRenderer` gestisce il rendering di ogni strumento personalizzato con un approccio generico: un corpo trascinabile (afferrabile dall'header), sonde posizionate sul lato sinistro del corpo, fili tratteggiati verso i pin connessi, e un display nell'header il cui contenuto varia in base al tipo di output.

Per gli strumenti con output numerico, il display mostra il valore letto dal pin connesso (tensione o resistenza, in base all'unità configurata), aggiornato in tempo reale. Per gli strumenti con output a LED, un indicatore luminoso cambia stato in base alla connessione e, opzionalmente, in base alla tensione rilevata.

Gli strumenti di tipo `firmware-dump` implementano un comportamento più complesso: quando tutte le sonde sono correttamente collegate, viene automaticamente innescata una modale con animazione di dump (scrolling esadecimale) e barra di progresso, al termine della quale viene offerto il download del file firmware.

3.4.3 Authoring degli strumenti personalizzati

Il pannello `CustomToolsPanel` nelle impostazioni offre un'interfaccia CRUD completa per la creazione e modifica degli strumenti. L'autore può aggiungere sonde con drag-and-drop dei colori, definire le modalità di misura, caricare un'immagine per il corpo dello strumento, e configurare il tipo di output.

Ogni strumento può essere duplicato — una funzionalità particolarmente utile per creare varianti dello stesso strumento — e l'ordine nell'elenco determina l'ordine di visualizzazione nella sidebar del simulatore.

3.5 Il pannello di authoring

Il pannello di authoring (`/settings`) rappresenta l'interfaccia con cui i docenti progettano gli esercizi. La sua struttura riflette il modello dati dell'esercizio, organizzando le funzionalità in sezioni tematiche accessibili tramite una sidebar laterale a tab.

3.5.1 Organizzazione dell'interfaccia

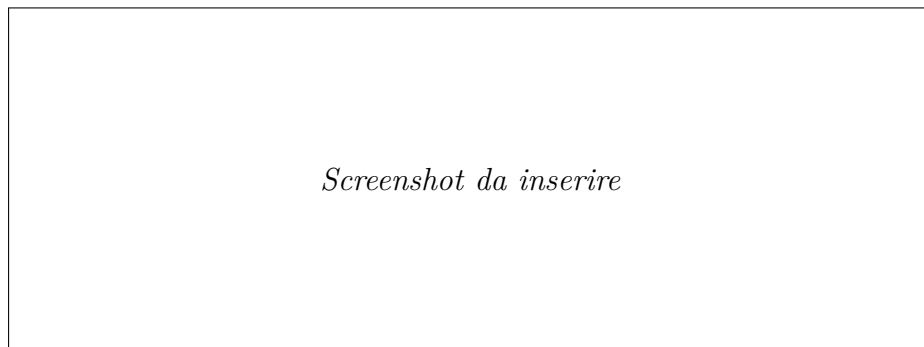


Figura 3.7: Pannello di authoring: a sinistra la sidebar con le tab Init/Challenge, al centro il canvas interattivo con la fotografia del PCB e gli overlay trascinabili dei componenti e dei pin.

La sidebar è divisa in due tab principali: **Init** e **Challenge**.

La tab **Init** contiene tutto ciò che riguarda la configurazione iniziale del circuito:

- **Componenti:** aggiunta e posizionamento dei componenti hardware sul PCB. L'autore seleziona un punto sull'immagine, definisce nome e tipo del componente, e il sistema crea un overlay cliccabile alle coordinate normalizzate corrispondenti.
- **Pin:** definizione dei punti di misura, dei pin UART e dei pin SPI. Per i pin di misura, l'autore specifica i valori elettrici (tensione, resistenza) con la possibilità di definire intervalli anziché valori fissi. Per i pin UART e SPI, l'autore assegna i ruoli corrispondenti.
- **Terminale:** editor completo per i componenti terminale, descritto in dettaglio nella sezione successiva.

- **Strumenti:** configurazione degli strumenti built-in (raggio e zoom della lente, parametri dell'adattatore UART, durata del dump firmware) e gestione dei gruppi di strumenti e degli strumenti personalizzati.

La tab **Challenge** contiene la definizione della struttura didattica dell'esercizio: gli step, i loro obiettivi, le condizioni di completamento e i frammenti di flag. Per ogni tipo di obiettivo è disponibile un popup di configurazione dedicato che espone i campi pertinenti — ad esempio, l'obiettivo terminale mostra un selettore per il componente terminale e le condizioni di boot stage, mentre l'obiettivo firmware-dump mostra la configurazione delle sonde SPI.

3.5.2 Il canvas interattivo

Il canvas delle impostazioni mostra l'immagine del PCB con sovrapposti gli handle trascinabili di componenti e pin. L'autore può zoomare, ruotare e traslare la vista per posizionare con precisione gli elementi, e i popup di configurazione degli obiettivi vengono renderizzati come overlay posizionati in modo assoluto alle coordinate dell'obiettivo selezionato.

Le trasformazioni del canvas (zoom, rotazione, traslazione) sono gestite interamente nello `settingsStore` e non influenzano le coordinate normalizzate degli elementi, che rimangono espresse in percentuale rispetto all'immagine non trasformata.

3.5.3 L'editor del terminale

L'editor del terminale, implementato nel componente `TerminalSettingsPanel`, offre un'interfaccia strutturata per la configurazione di tutti gli aspetti del sistema terminale. L'editor è organizzato in quattro sezioni:

- **Comandi:** elenco e editor dei comandi per ogni tab. Ogni comando è configurabile per tipo di handler (built-in, custom, dynamic, script), output, vincoli e side effect. Un popup dedicato (`CommandEditorPopup`) fornisce un form dettagliato con campi condizionali in base al tipo selezionato.
- **Flag:** definizione delle parti della flag terminale, ciascuna con identificativo, frammento di testo, descrizione e suggerimento. La flag completa è la concatenazione delle parti nell'ordine definito.
- **Boot:** configurazione degli stadi della sequenza di boot, con possibilità di riordinamento, definizione delle righe di output, durata e transizioni.

- **Filesystem:** editor ad albero del filesystem simulato, con supporto per la creazione di directory e file, modifica del contenuto, e assegnazione di permessi e proprietario.

A complemento dell’editor, un pannello di **anteprima** (`TerminalPreviewPanel`) mostra la configurazione in formato YAML — utile per una visione d’insieme — e un terminale funzionante in anteprima che permette all’autore di testare i comandi senza uscire dalle impostazioni.

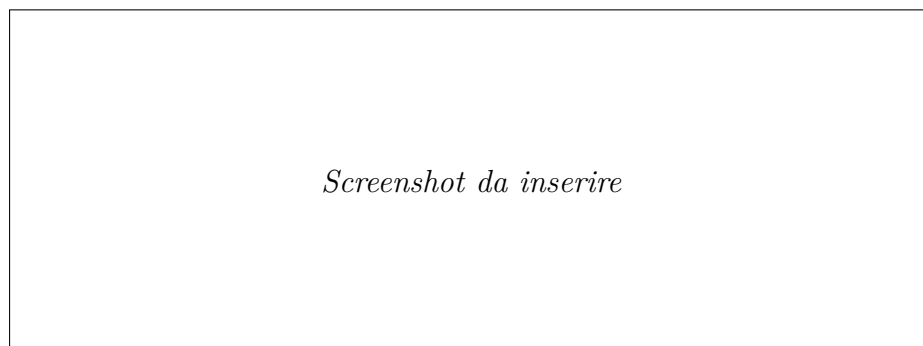


Figura 3.8: Editor del terminale nel pannello di authoring, con la sezione comandi e l’anteprima YAML della configurazione.

3.6 API e persistenza dei dati

3.6.1 Architettura delle API

Le API REST sono implementate come **Route Handler** di Next.js, un pattern che permette di definire endpoint HTTP direttamente nelle directory dell’applicazione senza un server separato. Ogni route handler è un file TypeScript che esporta funzioni corrispondenti ai metodi HTTP supportati.

Le API si dividono in quattro gruppi funzionali:

Configurazione dell’esercizio (`/api/config/`):

- `GET /api/config/load`: legge il file `exercise.override.json` dalla directory `src/data/` e lo restituisce come JSON. Se il file non esiste, restituisce un 404 che il client interpreta come assenza di configurazione personalizzata, ricadendo sui dati predefiniti.
- `POST /api/config/save`: riceve il JSON della configurazione e lo scrive su file, sovrascrivendo il contenuto precedente.

Configurazione del terminale (`/api/terminal-config/`):

- La struttura è analoga alla configurazione dell'esercizio. Il formato supporta sia una singola `TerminalConfig` sia un array di componenti terminale nel formato bundle, garantendo compatibilità sia con il vecchio che con il nuovo formato modulare.

Preset (`/api/presets/`):

- `GET /api/presets`: restituisce l'indice dei preset, letto dal file `index.json`.
- `POST /api/presets`: crea un nuovo preset. Il processo include la generazione di uno slug univoco dal nome, la copia dell'immagine PCB in un percorso dedicato al preset, la scrittura del file JSON del preset e l'aggiornamento dell'indice.
- `GET /api/presets/[id]`: carica un preset specifico dal suo file JSON.
- `PUT /api/presets/[id]`: aggiorna un preset esistente con la configurazione corrente.
- `DELETE /api/presets/[id]`: elimina il file del preset e rimuove la voce dall'indice.

Un modulo helper condiviso (`_helpers.ts`) centralizza le operazioni comuni: lettura e scrittura dell'indice, risoluzione del percorso di un preset, generazione degli slug, e copia delle immagini.

Media (`/api/images/`, `/api/firmware/`):

- `POST /api/images/upload`: gestisce l'upload multipart di immagini per gli strumenti personalizzati e i componenti, salvandole nella directory `public/uploads/`.
- `DELETE /api/images/delete`: rimuove un file immagine dato il suo percorso.
- `POST /api/firmware/upload`: gestisce l'upload di file firmware binari, salvandoli con un nome che include un timestamp per evitare collisioni.

3.6.2 Strategia di persistenza

L'intero sistema di persistenza si basa su **file JSON nel filesystem locale** e su **localStorage nel browser**. Questa scelta architetturale, motivata dalla natura didattica e single-user della piattaforma, elimina la complessità di un database e rende il progetto autocontenuto.

Il flusso di salvataggio segue un percorso che attraversa tre livelli:

1. **Stato in memoria** (store Zustand): rappresenta la versione corrente e modificabile della configurazione, aggiornata in tempo reale dalle interazioni dell'utente.
2. **localStorage**: funge da cache lato client. Ogni salvataggio scrive la configurazione serializzata in una chiave dedicata, permettendo il ripristino rapido al caricamento della pagina senza necessità di una chiamata API.
3. **File su disco**: la versione “canonica” della configurazione, scritta tramite le API REST. Questo livello garantisce la persistenza anche dopo la cancellazione del localStorage e permette di versionare le configurazioni tramite Git.

I preset aggiungono un quarto livello, raggruppando configurazione dell'esercizio e del terminale in un unico file che funge da snapshot riproducibile. La separazione tra configurazione “attiva” (quella su cui l'autore sta lavorando) e preset “salvati” (snapshot immutabili) consente un workflow flessibile: l'autore può sperimentare liberamente sapendo di poter sempre tornare a una versione salvata.

3.6.3 Sincronizzazione real-time

Un aspetto che merita attenzione è il meccanismo di sincronizzazione tra la modalità di authoring e il simulatore quando entrambi sono aperti in tab diverse del browser. L'hook `useTerminalConfig()` si sottoscrive all'evento `storage` del browser — emesso automaticamente quando una tab scrive in localStorage — e a un evento personalizzato `terminal-config-updated` per la sincronizzazione nella stessa tab. Quando viene rilevata una modifica, l'hook ricarica la configurazione e il terminale si aggiorna automaticamente.

Questo meccanismo rende il ciclo di sviluppo degli esercizi particolarmente fluido: l'autore può tenere aperte le impostazioni in una tab e il simulatore in un'altra, applicare una modifica alla configurazione del terminale e verificarne immediatamente l'effetto nel simulatore, senza ricaricare la pagina.

3.7 Il sistema di completamento dell'esercizio

3.7.1 Flussi di completamento degli obiettivi

Il completamento di un obiettivo segue percorsi diversi a seconda del tipo, ma converge in tutti i casi sull'azione interna `_completeCurrentObjective()` dello store:

- Gli **obiettivi di tipo component** si completano al click sullo hotspot corrispondente, verificato tramite il confronto tra l'identificativo del componente cliccato e quello atteso dall'obiettivo.
- Gli **obiettivi di tipo pin** si completano quando le connessioni delle sonde soddisfano le condizioni definite in `pinConditions`. Il sistema supporta sia la logica AND (tutte le condizioni devono essere soddisfatte) sia la logica OR (almeno una condizione è sufficiente), configurabile tramite il campo `pinLogic`.
- Gli **obiettivi di tipo uart** si completano quando tutte le connessioni richieste tra l'adattatore e il PCB sono state stabilite correttamente. La verifica avviene nell'azione `hookUartProbe()`, che ad ogni nuova connessione controlla se tutti i pin dell'adattatore sono collegati ai pin corrispondenti del circuito.
- Gli **obiettivi di tipo terminal** si completano quando tutte le flag elencate in `bootStageConditions.unlockedFlags` sono state scoperte all'interno del terminale. La verifica avviene nell'azione `addTerminalDiscovery()`, che dopo ogni nuova scoperta controlla se l'insieme delle flag richieste è un sottoinsieme delle flag effettivamente scoperte.
- Gli **obiettivi di tipo firmware-dump** seguono un percorso articolato in più fasi. Dopo il collegamento delle sonde e l'eventuale completamento dei comandi terminale (se configurato), lo studente avvia il dump e scarica il file. L'obiettivo si completa solo dopo il download, tramite l'azione `dismissFirmwareDumpDownload()`, che verifica lo stato corrente e decide se completare direttamente l'obiettivo o se lanciare un terminale per ulteriori analisi.

3.7.2 Reset dello stato alla transizione tra step

Un aspetto rilevante del flusso di completamento riguarda la **pulizia dello stato** alla transizione tra uno step e il successivo. Quando lo studente avanza al prossimo step — sia avviandolo dalla schermata educativa sia completando lo step corrente — il sistema esegue un reset completo di tutti gli strumenti: le sonde del multimetro vengono scollegate, le connessioni UART e SPI azzerate, il terminale chiuso, la lente disattivata, e la sidebar riportata allo strumento puntatore. Questo comportamento garantisce che ogni step inizi da uno stato pulito, evitando che connessioni residue dallo step precedente interferiscano con gli obiettivi successivi o generino confusione nello studente.

3.7.3 Progressione della flag

Durante l'esecuzione dell'esercizio, la flag viene costruita progressivamente. Ogni step completato contribuisce con il proprio `flagPart`; per gli step con obiettivi terminale, il contributo include sia il `flagPart` dell'obiettivo sia il `terminalCurrentFlag` — il frammento parziale della flag terminale, aggiornato in tempo reale man mano che lo studente scopre le singole parti.

L'interfaccia visualizza questa progressione in modo chiaro: i frammenti già scoperti appaiono in evidenza, quelli ancora da scoprire sono rappresentati da una sequenza di caratteri di sostituzione. Questo approccio fornisce allo studente un'indicazione del progresso senza rivelare il contenuto delle parti mancanti.

3.7.4 Il dialogo di completamento

Al completamento dell'ultimo step, il sistema verifica la flag complessiva e mostra un **dialogo di completamento** configurabile. Il dialogo può includere:

- un titolo e una descrizione personalizzati;
- un pulsante per copiare la flag negli appunti;
- un pulsante di redirect verso un URL esterno (ad esempio, una piattaforma di valutazione);
- un pulsante per scaricare un file (ad esempio, un certificato di completamento o un report).

La configurabilità del dialogo consente di adattare la piattaforma a diversi contesti d'uso — da un esercizio standalone con autocorrezione a un modulo integrato in un sistema di e-learning più ampio.

Capitolo 4

Caso di studio

Per verificare le capacità espressive della piattaforma e illustrarne concretamente l'uso, è stato progettato un esercizio CTF completo basato sull'analisi di un dispositivo reale: il router TP-Link WR841N, un dispositivo consumer ampiamente diffuso e frequentemente utilizzato nella ricerca sulla sicurezza dei dispositivi embedded.

La scelta di questo dispositivo non è casuale. Il WR841N è stato oggetto di numerosi studi e disclosure di vulnerabilità nel corso degli anni, il che lo rende un soggetto ideale per un esercizio didattico: è sufficientemente semplice da essere accessibile a studenti senza esperienza pregressa, ma presenta una superficie di attacco abbastanza ampia da consentire un percorso formativo articolato, che spazia dall'ispezione visiva del circuito stampato fino all'analisi delle vulnerabilità del software.

4.1 Progettazione dell'esercizio

4.1.1 Struttura dell'esercizio

L'esercizio è articolato in due step formali, corrispondenti alle fasi di analisi hardware, seguiti da una fase di esplorazione terminale che si attiva automaticamente al completamento della connessione UART:

Step 1 — Hardware Analysis. Lo studente deve identificare visivamente i componenti principali sulla fotografia del PCB del router: il System-on-Chip (SoC) QCA953x e la memoria EPROM. Successivamente, deve collegare le sonde del firmware dumper ai pin della memoria per estrarre il firmware. Questo step introduce le competenze fondamentali di ricognizione fisica e di interazione con i bus di comunicazione.

Step 2 — UART. Lo studente deve individuare i pin della porta seriale UART sul circuito e collegare l'adattatore USB-to-serial rispettando la logica del protocollo:

TX dell'adattatore su RX del dispositivo, RX su TX, GND su GND. Il completamento di questo step — che attesta la corretta comprensione del protocollo — innesca automaticamente l'apertura del terminale simulato, grazie alla configurazione `terminalComponentId` dell'adattatore UART.

Fase terminale (auto-lanciata). Il terminale, avviato automaticamente dopo la connessione UART, riproduce il comportamento del router TP-Link con una sequenza di boot realistica (U-Boot → kernel Linux → login). Il componente terminale è configurato con un sistema di sei flag progressive che guidano l'analisi:

1. *Boot Analysis* — analisi delle variabili d'ambiente del bootloader U-Boot tramite il comando `printenv`, alla ricerca di inconsistenze nella configurazione (ad esempio, `rootfstype=jffs2` nel bootloader mentre il kernel monta un filesystem SquashFS);
2. *Root Access* — ottenimento dell'accesso root tramite credenziali di default del dispositivo (`root` / `sohoadmin`);
3. *Hash Cracking* — esame del file `/etc/shadow` per estrarre gli hash delle password e utilizzo del tab “Local Machine” per crackarli con strumenti come `hashcat`;
4. *Data Leak* — analisi delle partizioni del dispositivo con il comando `strings` per estrarre credenziali WiFi e configurazioni in chiaro dalla partizione di configurazione;
5. *Command Injection* — analisi del binario del web server (`/usr/bin/httpd`) per individuare funzioni vulnerabili a injection di comandi;
6. *Backdoor Discovery* — identificazione di processi sospetti tramite `ps` e analisi di binari anomali in `/usr/bin/`.

Ogni scoperta sblocca un frammento della flag terminale; al completamento di tutti e sei, la flag composta `flag{b00t_r00t_h4sh_134k_1nj3ct_sh3ll}` viene rivelata. Questa architettura — step formali per l'interazione hardware, fase terminale auto-lanciata per l'analisi software — sfrutta il meccanismo di auto-lancio descritto nel Capitolo 4 (§4.2.2) per collegare fluidamente le due fasi dell'esercizio.

4.1.2 Configurazione del terminale

Il terminale dell'esercizio è configurato con due tab:

- **UART Console:** simula la connessione seriale al dispositivo. Include una sequenza di boot completa che riproduce fedelmente l'output del bootloader U-Boot del QCA953x (banner, inizializzazione RAM, checksum del kernel) seguito dal boot Linux con i messaggi del kernel. Il filesystem replica la struttura reale del WR841N: directory `/etc` con `passwd`, `shadow`, `inittab`; `/proc` con `cpuinfo`, `mtd`, `meminfo`; `/web/oem/model.conf` con il modello e la versione del firmware.
- **Local Machine:** simula la macchina di analisi dello studente, con strumenti come `hashcat` e `john` disponibili per il cracking degli hash estratti dal dispositivo target.

4.1.3 Configurazione degli strumenti

L'esercizio utilizza la seguente configurazione degli strumenti:

- La **lente di ingrandimento** è configurata con raggio 90px e zoom 2.4x, parametri calibrati sulla risoluzione della fotografia del PCB utilizzata.
- L'**adattatore UART** è configurato con persistenza post-connessione e con associazione al componente terminale, in modo che il terminale si apra automaticamente dopo il collegamento.
- Il **firmware dumper** utilizza due sonde (GND e VCC) con mapping esatto verso pin specifici, durata del dump di 1 secondo, e un file firmware binario reale (`fw_test.bin`) scaricabile dallo studente.
- Il **dialogo di completamento** mostra un messaggio di congratulazioni con la possibilità di copiare la flag.

4.2 Walkthrough dello studente

Si descrive di seguito il percorso tipico di uno studente che affronta l'esercizio, evidenziando le interazioni con la piattaforma in ciascuna fase.

4.2.1 Fase 1 — Ricognizione del PCB

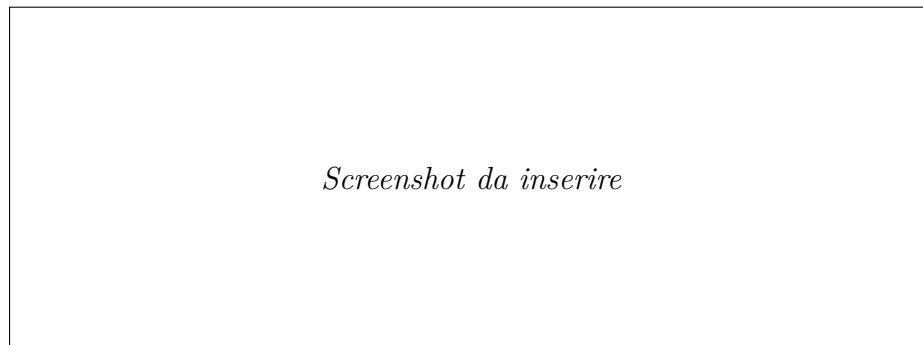


Figura 4.1: Schermata educativa del primo step, con la descrizione dell’esercizio e il pulsante per avviare la fase attiva.

Lo studente accede alla piattaforma e visualizza la schermata educativa del primo step, che introduce il contesto dell’esercizio. Dopo aver letto le istruzioni, avvia lo step e si trova di fronte alla fotografia del PCB del router.

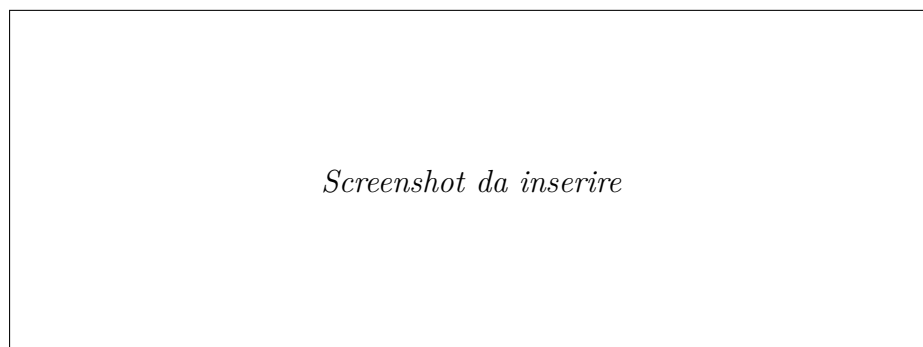


Figura 4.2: PCB del TP-Link WR841N nel simulatore, con la lente di ingrandimento posizionata sul SoC QCA953x e il componente evidenziato dopo l’identificazione.

Utilizzando la lente di ingrandimento, esamina il circuito alla ricerca dei componenti descritti nelle istruzioni. Identifica il SoC cliccando sull’area corrispondente — il sistema evidenzia il componente e rivela il primo frammento della flag. Ripete l’operazione per la memoria EPROM.

Per il terzo obiettivo dello step, seleziona lo strumento firmware dump dalla sidebar. Due sonde appaiono sullo schermo; lo studente le trascina sui pin indicati. Quando entrambe le sonde sono correttamente collegate (il meccanismo di snap facilita il posizionamento), il pulsante “DUMP FIRMWARE” si abilita. Avvia il dump, osserva la barra di progresso, e al termine scarica il file binario del firmware. L’ultimo frammento della flag dello step viene rivelato.

4.2.2 Fase 2 — Connessione UART

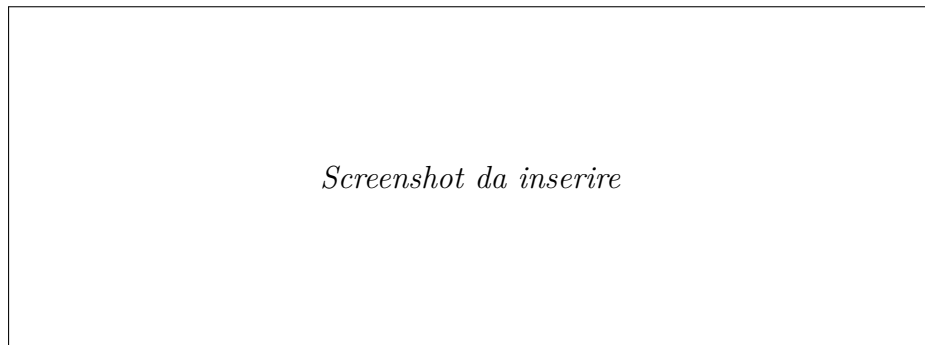


Figura 4.3: Adattatore UART con le sonde collegate ai pin del PCB: la connessione incrociata TX→RX è evidenziata dai fili colorati.

Nel secondo step, lo studente deve stabilire la connessione seriale. Seleziona lo strumento UART e trascina le sonde dell’adattatore sui pin del PCB. Se tenta di collegare TX su TX, la connessione viene rifiutata — un feedback immediato che lo invita a riflettere sulla logica del protocollo. Dopo aver individuato il mapping corretto (TX→RX, RX→TX, GND→GND), la connessione viene stabilita e il terminale si apre automaticamente.

4.2.3 Fase 3 — Esplorazione del terminale

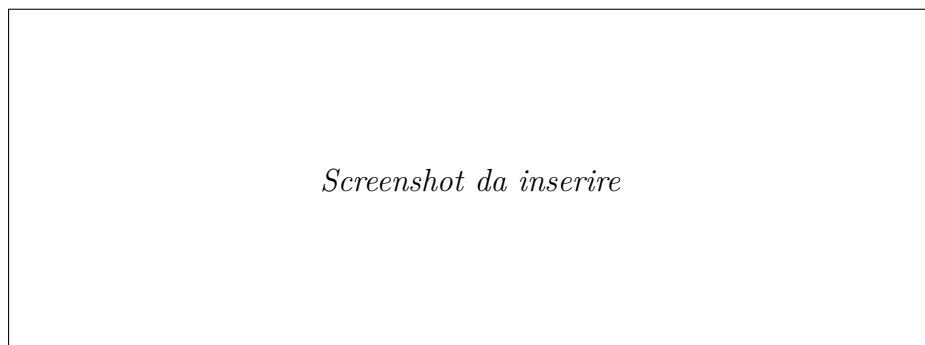


Figura 4.4: Terminale simulato durante la sequenza di boot del WR841N, con i messaggi del bootloader U-Boot e l’inizializzazione del kernel Linux.

Il terminale mostra la sequenza di boot del dispositivo: il banner U-Boot, l’inizializzazione dell’hardware, il caricamento del kernel. Lo studente osserva il flusso dei messaggi, che replica fedelmente quello di un dispositivo reale.

Arrivato al prompt U-Boot (`U-Boot>`), esegue il comando `printenv` per esaminare le variabili d’ambiente del bootloader. Nota che `rootfstype=jffs2` mentre il

kernel successivamente monta un filesystem SquashFS — un’inconsistenza che suggerisce una modifica non autorizzata. Questo sblocca il primo frammento della flag terminale.

Dopo il boot del kernel, il dispositivo presenta un prompt di login. Lo studente prova le credenziali di default: `root` come utente e `sohoadmin` come password. L’accesso viene concesso e il secondo frammento viene sbloccato.

Con l’accesso `root`, lo studente esamina il file `/etc/shadow` per estrarre gli hash delle password. Passa al tab “Local Machine” e utilizza `hashcat` per crackare l’hash MD5. Scopre la password in chiaro e sblocca il terzo frammento.

Prosegue con l’analisi delle partizioni: `strings /dev/mtdblock3` rivela credenziali WiFi e configurazioni in chiaro nella partizione di configurazione — un classico esempio di data leak in dispositivi embedded. Quarto frammento sbloccato.

L’analisi del web server (`strings /usr/bin/httpd | grep exec`) rivela la presenza di funzioni vulnerabili a command injection come `execFormatCmd`. Quinto frammento.

Infine, il comando `ps` rivela un processo sospetto. L’analisi del binario `/usr/bin/backdoorTest` con `strings` e `file` ne conferma la natura malevola. Sesto e ultimo frammento sbloccato.

La flag terminale è ora completa; il sistema combina i frammenti degli step precedenti con quelli del terminale per costruire la flag finale dell’esercizio. Lo studente la inserisce nel dialogo di validazione e riceve la conferma del completamento.

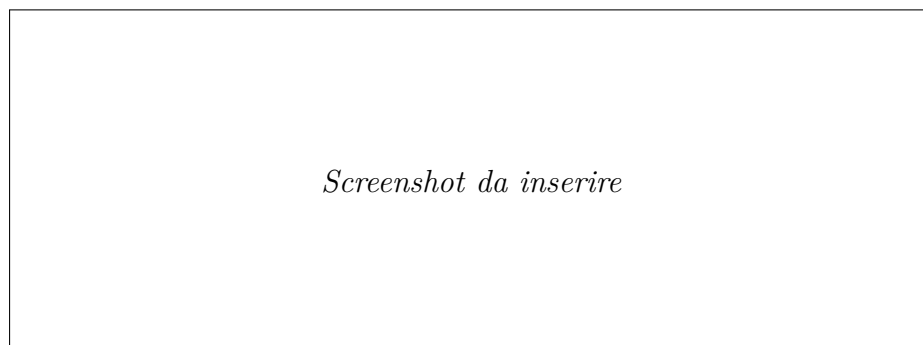


Figura 4.5: Terminale con la flag progressiva completata e il dialogo di completamento dell’esercizio con la flag finale.

4.3 Configurazione lato autore

4.3.1 Creazione dell'esercizio



Figura 4.6: Pannello di authoring con la fotografia del PCB del WR841N e i componenti posizionati tramite drag-and-drop.

La configurazione dell'esercizio descritto è stata interamente realizzata tramite il pannello di authoring della piattaforma, senza modifiche dirette ai file JSON.

Il processo è partito dal caricamento della fotografia del PCB nella tab "Image" delle impostazioni. L'autore ha poi posizionato i componenti cliccabili (SoC, EPROM) direttamente sull'immagine tramite drag-and-drop, regolando le dimensioni degli overlay fino a coprire le aree corrette del circuito.

I pin di misura e i pin UART sono stati posizionati in modo analogo, ciascuno con il ruolo e i valori elettrici appropriati. Per il firmware dump, l'autore ha configurato le sonde e caricato il file binario tramite l'apposito pannello.

4.3.2 Configurazione del terminale

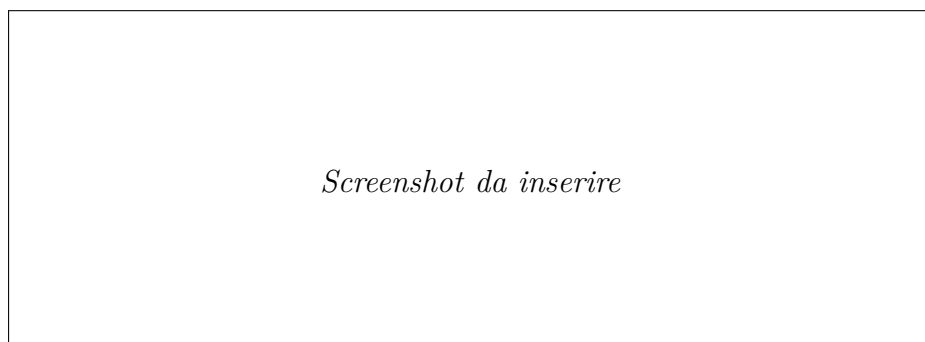


Figura 4.7: Editor del terminale con la configurazione dei comandi del tab UART Console e la struttura del filesystem del WR841N.

La configurazione del terminale ha rappresentato la parte più laboriosa. L'autore ha utilizzato l'editor strutturato per:

- definire la struttura del filesystem tab per tab, creando le directory e i file che replicano il sistema del WR841N;
- configurare i comandi built-in (`ls`, `cd`, `cat`, `pwd`, `grep`, `find`, `clear`) con i vincoli di boot stage appropriati — ad esempio, i comandi del filesystem sono disponibili solo nello stadio `shell`, non nel prompt U-Boot;
- creare i comandi custom (`printenv`, `version`, `strings`, `ps`, `hashcat`) con i rispettivi output statici, condizionali o lookup;
- definire i side effect di sblocco flag per ciascun comando chiave;
- configurare la sequenza di boot con gli stadi U-Boot, kernel, login e shell.

L'anteprima integrata ha permesso di testare iterativamente il comportamento del terminale durante la configurazione, verificando che i comandi producano l'output atteso e che le flag si sbloccino correttamente.

4.3.3 Salvataggio come preset

Al termine della configurazione, l'esercizio è stato salvato come preset con il nome "CTF-1" e la descrizione "TP-LINK". Il sistema ha creato automaticamente una copia dell'immagine del PCB dedicata al preset, garantendo che l'esercizio sia riproducibile anche se l'immagine di lavoro viene successivamente modificata.

Il preset così creato può essere caricato in qualsiasi istanza della piattaforma, permettendo al docente di distribuire l'esercizio ai propri studenti con un semplice file JSON, oppure di condividerlo con colleghi che utilizzano la stessa piattaforma.

Capitolo 5

Validazione e discussione

5.1 Criteri di valutazione

La validazione di una piattaforma didattica presenta sfide metodologiche specifiche: l'efficacia formativa è intrinsecamente difficile da misurare in modo quantitativo, poiché dipende da variabili quali il livello di partenza degli studenti, il contesto di utilizzo e la qualità degli esercizi proposti. In assenza di un campione statisticamente significativo di utenti — la piattaforma è stata sviluppata nell'ambito di questo lavoro di tesi e non è ancora stata sottoposta a una sperimentazione su larga scala — la validazione si articola su tre assi qualitativi: la copertura dei concetti didattici, la fedeltà della simulazione rispetto all'esperienza reale, e l'espressività del sistema di authoring.

5.1.1 Copertura dei concetti didattici

Il percorso formativo coperto dalla piattaforma abbraccia le fasi principali dell'analisi hardware di un dispositivo embedded:

Tabella 5.1: Copertura dei concetti didattici nelle fasi dell’analisi hardware

Fase dell’analisi	Strumento/funzionalità	Concetto didattico
Ricognizione fisica	Lente di ingrandimento, click su componenti	Identificazione dei chip, lettura delle sigle
Misurazioni elettriche	Multimetro (tensione, resistenza)	Individuazione dei livelli logici, test di continuità
Identificazione interfacce	Pin UART (TX, RX, GND)	Riconoscimento dei bus di comunicazione seriale
Connessione seriale	Adattatore UART con crossover	Protocollo UART, logica TX/RX incrociata
Estrazione firmware	Sonde SPI (6 segnali)	Bus SPI, ruoli dei segnali, dump della memoria flash
Analisi bootloader	Terminale con boot sequence	U-Boot, variabili d’ambiente, configurazione del boot
Accesso al sistema	Login con credenziali	Credenziali di default, hardening insufficiente
Analisi del filesystem	Comandi built-in (<code>ls</code> , <code>cat</code> , <code>grep</code> , <code>find</code>)	Struttura di un sistema Linux embedded
Cracking di password	Tab “Local Machine” con hashcat	Hash MD5, attacchi dizionario, <code>/etc/shadow</code>
Analisi delle vulnerabilità	Comandi <code>strings</code> , <code>ps</code>	Command injection, backdoor, data leak

Questa copertura corrisponde al percorso tipico descritto nei testi di riferimento sull’analisi hardware, come “The Hardware Hacking Handbook” (Jasper van Woudenberg, Colin O’Flynn) e “Practical IoT Hacking” (Fotios Chantzis et al.), coprendo le fasi dalla ricognizione fisica fino all’analisi delle vulnerabilità software.

Un’area non coperta dalla piattaforma nella sua versione attuale è quella degli attacchi side-channel (analisi di consumo energetico, emissioni elettromagnetiche, timing attack) e del protocollo JTAG, che richiederebbero un livello di simulazione significativamente più complesso e che sono tipicamente affrontati in fasi avanzate della formazione.

5.1.2 Fedeltà della simulazione

La simulazione implementata opera necessariamente a un livello di astrazione superiore rispetto all’interazione con hardware reale. Tuttavia, diversi accorgimenti contribuiscono a ridurre il divario percettivo:

- Il **rumore digitale** del multimetro riproduce l'instabilità tipica delle ultime cifre di uno strumento reale, insegnando implicitamente allo studente a interpretare le letture con la corretta incertezza.
- La **validazione basata sui ruoli** delle connessioni UART e SPI obbliga lo studente a ragionare sulla logica dei protocolli, non semplicemente a “indovinare” il collegamento corretto.
- La **sequenza di boot** del terminale replica fedelmente l'output di un dispositivo reale, con tempi di animazione calibrati per risultare credibili.
- Il **filesystem** include file e directory coerenti con il sistema operativo del dispositivo simulato, non placeholder generici.

Il limite principale riguarda l'assenza di variabilità: in un dispositivo reale, le misurazioni sono soggette a rumore ambientale, i collegamenti possono essere intermittenti, e il comportamento del sistema può variare tra sessioni. La simulazione, per sua natura deterministica, non cattura questa complessità. Questa è una limitazione consapevole, non un difetto: in fase didattica, il determinismo è un vantaggio, poiché consente allo studente di concentrarsi sui concetti senza le frustrazioni legate a problemi di contatto o interferenze.

5.1.3 Espressività del sistema di authoring

Il sistema di authoring è stato progettato per consentire la creazione di esercizi eterogenei senza modifiche al codice sorgente. Per valutarne l'espressività, consideriamo la varietà di scenari configurabili:

- **Esercizi di sola ricognizione:** step con obiettivi di tipo “component”, dove lo studente identifica i chip utilizzando la lente di ingrandimento. Non richiedono terminale né connessioni.
- **Esercizi di misura elettrica:** step con obiettivi di tipo “pin”, dove lo studente utilizza il multimetro per individuare livelli logici o verificare continuità.
- **Esercizi di connessione:** step con obiettivi UART o firmware-dump, focalizzati sulla comprensione dei protocolli di comunicazione.
- **Esercizi di analisi terminale:** step con obiettivi terminale complessi, sequenze di boot multi-stadio, flag condizionali.

- **Esercizi combinati:** step che integrano strumenti fisici e terminale, come nell'esercizio del caso di studio.

Il sistema di sei tipi di output del terminale (statico, condizionale, template, lookup, dinamico, script) consente di simulare una gamma molto ampia di comportamenti, dalla semplice restituzione di testo fisso fino all'esecuzione di logica arbitraria via funzioni JavaScript.

I vincoli sui comandi (percorso, permessi, prerequisiti, argomenti, stadio di boot) permettono di costruire percorsi didattici guidati dove la disponibilità dei comandi evolve con il progresso dello studente, riducendo il rischio che salti fasi o arrivi a risultati senza aver compreso i passaggi intermedi.

5.2 Confronto con soluzioni esistenti

Per contestualizzare il contributo di PCB-CTF, è utile confrontarlo con le piattaforme e gli approcci didattici esistenti nel campo della sicurezza hardware.

Tabella 5.2: Confronto tra PCB-CTF e le soluzioni esistenti

Caratteristica	CTFd / PicoCTF	RHme (Riscure)	Remote Labs	PCB-CTF
Simulazione hardware	No	No (hardware reale)	Parziale	Sì
Interazione con PCB	No	Sì (fisico)	Via webcam	Sì (virtuale)
Strumenti simulati	No	No (reali)	Reali via rete	Sì
Terminale embedded	No	No	Sì (SSH)	Sì (simulato)
Accessibilità browser	Sì	No	Parziale	Sì
Costo per studente	Zero	Hardware (~ € 50+)	Infrastruttura	Zero
Scalabilità	Alta	Bassa	Media	Alta
Authoring no-code	Parziale	No	No	Sì
Determinismo	N/A	No	No	Sì
Sicurezza server	N/A	N/A	Rischio	Nessun rischio

CTFd e PicoCTF sono piattaforme CTF general-purpose, eccellenti per sfide software (web, crypto, reverse engineering) ma prive di qualsiasi componente di interazione hardware. Lo studente affronta sfide di analisi firmware come esercizi di reverse engineering puro, senza il contesto dell'hardware che lo ospita.

RHme (Riscure Hack me) è una board hardware dedicata alla formazione sulla sicurezza, che richiede l'acquisto fisico del dispositivo. Offre un'esperienza autentica ma con costi e logistica che ne limitano la scalabilità.

I laboratori remoti (come quelli offerti da alcune università e organizzazioni di formazione) forniscono accesso a hardware reale via rete, ma richiedono infrastruttura dedicata, manutenzione, e presentano limiti di concorrenza e disponibilità.

PCB-CTF si posiziona in una nicchia non coperta dalle soluzioni esistenti: offre un'interazione visiva e strumentale con l'hardware — assente nelle piattaforme CTF tradizionali — in un formato completamente virtualizzato che non richiede hardware fisico né infrastruttura server complessa.

5.3 Limitazioni

Nonostante i risultati raggiunti, la piattaforma presenta limitazioni che è necessario riconoscere.

Assenza di esecuzione reale. Il terminale simula, non esegue. Lo studente non può lanciare comandi arbitrari, ma solo quelli previsti dall'autore. Questo implica che la piattaforma non può replicare l'esperienza esplorativa aperta tipica di un'analisi reale, dove il percorso è spesso non lineare e guidato dall'intuizione. Per mitigare questa limitazione, il sistema di output condizionale e lookup consente all'autore di prevedere un insieme ragionevolmente ampio di input, ma la copertura non potrà mai essere completa.

Scenari non coperti. Come già accennato, i protocolli JTAG, I²C, e CAN non sono attualmente supportati. Gli attacchi side-channel — analisi di potenza, timing, fault injection — richiederebbero un paradigma di simulazione fondamentalmente diverso e non rientrano nell'ambito di questo lavoro.

Singolo utente. La piattaforma non supporta sessioni concorrenti con autenticazione, scoring, o classifiche. In un contesto di competizione CTF, queste funzionalità sarebbero essenziali.

Persistenza limitata. Il progresso dello studente non viene salvato tra le sessioni. Questo è coerente con l'uso in laboratorio, ma limita l'adozione in contesti di e-learning dove lo studente potrebbe voler riprendere un esercizio in un momento successivo.

Validazione empirica. La piattaforma non è ancora stata sottoposta a una sperimentazione formale con studenti. La valutazione presentata in questo capitolo è qualitativa e basata sull'analisi delle funzionalità; una validazione quantitativa dell'efficacia didattica richiederebbe uno studio controllato con gruppi di studenti, pre-test e post-test, che costituisce un naturale sviluppo futuro.

5.4 Sviluppi futuri

I limiti identificati suggeriscono diverse direzioni di evoluzione:

Supporto multi-utente e scoring. L'aggiunta di un sistema di autenticazione e di una classifica trasformerebbe la piattaforma da strumento didattico individuale a piattaforma CTF competitiva, adatta a eventi e competizioni. Questo richiederebbe l'introduzione di un database per la persistenza degli utenti e dei punteggi, e una logica di timing per la classifica.

Integrazione con hardware reale. Un'evoluzione particolarmente interessante sarebbe la possibilità di alternare fasi simulate e fasi su hardware reale all'interno dello stesso esercizio. Ad esempio, lo studente potrebbe completare la ricognizione e le misurazioni in simulazione, e poi passare a un laboratorio remoto per l'estrazione fisica del firmware. Questo approccio ibrido combinerebbe i vantaggi della simulazione (sicurezza, scalabilità, determinismo) con l'autenticità dell'esperienza su hardware reale.

Nuovi protocolli e strumenti. L'architettura a strumenti personalizzati è già predisposta per l'estensione. Protocolli come JTAG e I²C potrebbero essere aggiunti come nuovi tipi di pin e strumenti, riutilizzando il meccanismo di connessione role-based e il sistema di validazione esistente.

Terminale con LLM. Un'evoluzione del terminale potrebbe integrare un modello linguistico per gestire comandi non previsti dall'autore, generando risposte plausibili e coerenti con il contesto simulato. Questo amplierebbe significativamente l'esperienza esplorativa dello studente, pur mantenendo il controllo sull'output dei comandi chiave per la progressione dell'esercizio.

Persistenza del progresso e analytics. Il salvataggio del progresso dello studente, combinato con analytics sull'utilizzo (tempo per obiettivo, comandi tentati, errori più frequenti), fornirebbe dati preziosi sia allo studente — per riprendere il lavoro — sia al docente — per identificare i punti di difficoltà e migliorare gli esercizi.

Conclusioni

Il presente lavoro ha affrontato il problema della formazione sulla sicurezza hardware in contesti dove l'accesso a laboratori fisici è limitato, proponendo una piattaforma web che simula l'intero flusso di lavoro dell'analisi di un dispositivo embedded — dall'ispezione visiva del circuito stampato fino all'estrazione e all'analisi del firmware — in un ambiente completamente virtualizzato e accessibile da browser.

La piattaforma PCB-CTF è stata progettata attorno a due principi guida: la **fedeltà dell'interazione**, per garantire che l'esperienza dello studente sia trasferibile al contesto reale, e la **configurabilità dichiarativa**, per consentire ai docenti di creare esercizi personalizzati senza modificare il codice sorgente.

Sul piano dell'interazione, gli strumenti simulati — multimetro con rumore digitale, adattatore UART con validazione incrociata, programmatore SPI con sei sonde role-based — replicano la logica operativa delle controparti reali. Il terminale, pur operando su dati predefiniti, riproduce in modo convincente il comportamento di un sistema embedded, dalla sequenza di boot del bootloader U-Boot fino all'esplorazione del filesystem Linux. Il meccanismo di coordinate normalizzate e snap automatico delle sonde rende l'interazione fluida e intuitiva, senza sacrificare il realismo.

Sul piano dell'autoring, il pannello di configurazione consente di definire esercizi arbitrariamente complessi attraverso un editor visuale. Il sistema di sei tipi di output del terminale, combinato con i vincoli contestuali sui comandi e il meccanismo di flag progressive, offre un livello di espressività sufficiente a replicare scenari di analisi realistici, come dimostrato dal caso di studio basato sul router TP-Link WR841N.

Le scelte architetturali si sono rivelate adeguate alla complessità del sistema. La gestione dello stato tramite quattro store Zustand indipendenti ha garantito una separazione chiara delle responsabilità, senza i costi di complessità di un sistema centralizzato. L'assenza di un database e l'adozione di file JSON per la persistenza hanno mantenuto il progetto leggero e immediatamente deployabile, coerentemente con la natura didattica dell'applicazione.

Il caso di studio ha dimostrato che la piattaforma è in grado di supportare un percorso formativo completo, dalla ricognizione hardware all'analisi delle vulnerabi-

lità software, in un formato guidato e gamificato. Lo studente attraversa le stesse fasi concettuali di un'analisi reale — identificare, misurare, connettersi, estrarre, esplorare, scoprire — acquisendo competenze metodologiche che costituiscono una base solida per il successivo approccio all'hardware fisico.

Le limitazioni individuate — l'assenza di esecuzione reale nel terminale, il mancato supporto di protocolli come JTAG e I²C, la mancanza di multi-utenza e persistenza del progresso — indicano chiaramente le direzioni di evoluzione futura. In particolare, l'integrazione con laboratori remoti per un approccio ibrido simulazione-hardware, e l'introduzione di un sistema di scoring per contesti competitivi, rappresentano le estensioni più promettenti.

In definitiva, PCB-CTF colma un vuoto nell'offerta formativa sulla sicurezza hardware, offrendo un'alternativa accessibile e scalabile ai laboratori fisici. Non si propone di sostituire l'esperienza su hardware reale, ma di preparare lo studente ad affrontarla con le competenze concettuali e la metodologia già acquisite, riducendo la barriera d'ingresso a un campo che, per la crescente pervasività dei dispositivi IoT, è destinato a diventare sempre più rilevante.

Appendice A

Struttura JSON di un esercizio completo

Si riporta di seguito lo schema completo della struttura dati **Exercise**, che rappresenta la configurazione di un esercizio nella piattaforma PCB-CTF. Per ciascun campo viene indicato il tipo, l'obbligatorietà e una descrizione sintetica del ruolo all'interno del sistema.

Lo schema è presentato in formato annotato, dove ogni campo è accompagnato da un commento esplicativo. I valori tra parentesi angolari (<...>) indicano il tipo atteso; i campi contrassegnati con ? sono opzionali.

A.1 Schema dell'esercizio (Exercise)

```
{
  // -- Immagine del PCB --
  "pcbImage": "<string>",
  // Percorso dell'immagine del circuito stampato (relativo a /
  // public).
  // Esempio: "/images/pcb_v2.jpg"

  // -- Flag iniziale --
  "initialFlag": "<string>",
  // Template della flag visualizzato allo studente prima che
  // inizi a
  // scoprire i frammenti. I caratteri '?' vengono
  // progressivamente
  // sostituiti dai flagPart degli obiettivi completati.
  // Esempio: "flag{????????????????}"
}
```



```
// -- Step dell'esercizio --
"steps": [
  {
    "id": "<string>",
    // Identificatore univoco dello step.
    // Generato con il pattern: "step-" + timestamp base36 +
    // random.

    "title": "<string>",
    // Titolo dello step, visualizzato nell'intestazione del
    // simulatore.

    "description": "<string>",
    // Testo descrittivo mostrato nella schermata educativa
    // iniziale
    // dello step, prima che lo studente avvii la fase attiva.

    "expectedFlag": "<string>",
    // Flag completa attesa per questo step. Viene confrontata
    // con la
    // concatenazione dei flagPart degli obiettivi per
    // verificare il
    // completamento. Esempio: "flag{CPU-123EPROMDUMP}"

    "availableTools?": ["<Tool>"],
    // Lista degli strumenti disponibili durante lo step.
    // Valori ammessi: "pointer", "magnifier", "multimeter",
    // "probes", "firmware-dump", "custom".
    // Se omissso, tutti gli strumenti sono disponibili.

    "objectives": [
      {
        "id": "<string>",
        // Identificatore univoco dell'obiettivo.

        "name": "<string>",
        // Nome dell'obiettivo, visualizzato nel pannello
        // laterale
        // e nel tracker di progressione.

        "type?": "<ObjectiveType>",
        // Tipologia dell'obiettivo. Determina il meccanismo
        // di
        // completamento. Valori ammessi:
        // "component" -- click su un componente del PCB
      }
    ]
  }
]
```

```
// "pin"          -- connessione sonde a pin
//                specifici
// "uart"         -- connessione UART corretta (
//                crossover)
// "terminal"     -- scoperta di flag nel terminale
// "firmware-dump" -- dump firmware SPI completato
// Se omissso, il tipo e' "component" (default
//                implicito).

"componentId?": "<string>",
// Per type="component": ID del componente hardware
//                collegato.
// Il click sull'overlay del componente completa l'
//                obiettivo.

"instruction": "<string>",
// Istruzione testuale mostrata allo studente quando
// l'obiettivo e' attivo.

"hint": "<string>",
// Suggerimento opzionale, rivelabile dallo studente.

"flagPart": "<string>",
// Frammento di flag sbloccato al completamento dell'
//                obiettivo.
// La concatenazione ordinata dei flagPart di tutti
//                gli
//                obiettivi di uno step deve corrispondere a
//                expectedFlag.

"coords": ["<number>", "<number>", "<number>", "<
//                number>"],
// Coordinate normalizzate [x%, y%, width%, height%]
//                dell'area
// cliccabile sul PCB. Per obiettivi non spaziali (
//                terminal,
//                uart) si usa [0, 0, 0, 0].

// -- Campi specifici per type="pin" / type="uart" --

"pinConditions?": [
{
    "pinId": "<string>",
    // ID del pin sul PCB che deve essere collegato.
```

```
"terminal": "<string>"
// Identificatore della sonda che deve collegare
// il pin.
// Per UART: "adapter-tx", "adapter-rx", "adapter-
// gnd"
// Per firmware-dump: "fw-probe:<probeId>"
// Per multimetro: "probe1", "probe2"
}
],

"pinLogic?": "<'AND' | 'OR'>",
// Operatore logico tra le pinConditions.
// "AND" (default): tutte le condizioni devono essere
// soddisfatte.
// "OR": almeno una condizione e' sufficiente.

// -- Campi specifici per type="terminal" --

"bootStageConditions?": [
{
  "bootStageId": "<string>",
  // ID dello stadio di boot associato (informativo)
  .

  "unlockedFlags": [<string>],
  // Lista di ID flag del terminale che devono
  // essere
  // scoperte per completare l'obiettivo. L'
  // obiettivo
  // si completa quando TUTTI i flag elencati sono
  // stati
  // sbloccati tramite i sideEffects dei comandi.

  "hint": "<string>"
  // Suggerimento per lo stadio (informativo).
}
],

"terminalComponentId?": "<string>",
// ID del componente terminale da attivare quando
// questo
// obiettivo diventa corrente.

"requiresUart?": "<boolean>",
```

```

        // Se true, il terminale non si avvia finche' la
        // connessione
        // UART non e' stata stabilita.

        "terminalPersistent?": "<boolean>",
        // Se true, il terminale non puo' essere chiuso dalla
        // toolbar
        // una volta attivato per questo obiettivo.

        // -- Campi specifici per type="firmware-dump" --

        "customToolId?": "<string>"
        // ID del custom tool di tipo firmware-dump collegato
        // all'obiettivo.
    }
}
],

// -- Componenti hardware cliccabili --
"components": [
    {
        "id": "<string>",
        "name": "<string>",
        "instruction": "<string>",
        "hint": "<string>",
        "flagPart": "<string>",
        "coords": ["<number>", "<number>", "<number>", "<number>"]
        // Stessa semantica dell'obiettivo: overlay cliccabile sul
        // PCB
        // con coordinate normalizzate in percentuale.
    }
],

// -- Pin di misura --
"pins": [
    {
        "id": "<string>",
        "valueV": "<number>", // Tensione in Volt letta dal
        // multimetro
        "valueOhm": "<number>", // Resistenza in Ohm letta dal
        // multimetro
        "coords": ["<number>", "<number>", "<number>", "<number>"]
    }
],

```

```

// -- Pin UART --
"uartPins": [
  {
    "id": "<string>",
    "role": "<UartRole>",          // "tx" | "rx" | "gnd" | "vcc"
    "label": "<string>",          // Etichetta visiva, es. "TX (3.3
                                V)"
    "coords": [<number>, <number>, <number>, <number>]
  }
],

// -- Pin SPI per firmware dump (opzionale) --
"firmwareDumpPins?": [
  {
    "id": "<string>",
    "role": "<SpiRole>",          // "vcc" | "gnd" | "cs" | "clk" |
                                "mosi" | "miso"
    "label": "<string>",
    "coords": [<number>, <number>, <number>, <number>]
  }
],

// -- Strumenti personalizzati (opzionale) --
"customTools?": [
  {
    "id": "<string>",
    "name": "<string>",
    "description?": "<string>",
    "probes": [
      {
        "id": "<string>",
        "label": "<string>",          // Etichetta visiva, es.
                                "CH1"
        "role": "<string>",          // Ruolo semantico, es.
                                "positive"
        "color": "<string>",          // Colore hex, es. "#22
                                C55E"
        "connectivity": "<string>"    // "measurement" | "uart
                                " | "all"
      }
    ]
  },
  "outputType": "<string>",
  // Tipo di visualizzazione nel simulatore:
  // "none" -- nessun display

```

```

//      "numeric"           -- valore numerico (tensione/
//                          resistenza)
//      "leds"              -- indicatori LED
//      "connection-status" -- stato della connessione
//      "firmware-dump"     -- interfaccia firmware dump

"outputUnit?": "<string>",      // Unità' per outputType
                              ="numeric"
"modes?": [
  {
    "id": "<string>",
    "name": "<string>",          // "Tensione"
    "shortName": "<string>",     // "V"
    "unit": "<string>"           // "V", "ohm", "Hz"
  }
],
"firmwareDumpConfig?": {
  "requiredConnections": [
    { "probeId": "<string>", "pinId": "<string>" }
  ],
  "filePath?": "<string>",
  "fileName?": "<string>",
  "dumpDurationSec?": "<number>"
}
],

// -- Percorso firmware caricato (opzionale) --
"firmwarePath?": "<string>",
// Percorso del file firmware sul server, usato dal firmware
// dumper
// built-in. Esempio: "/uploads/firmware-1772839791936-fw_test
// .bin"

// -- Gruppi di strumenti (opzionale) --
"toolGroups?": [
  {
    "id": "<string>",
    "name": "<string>",
    "toolIds": ["<string>"]
    // ID degli strumenti che possono essere attivi
    // contemporaneamente.
    // Gli strumenti nello stesso gruppo coesistono; strumenti
    // non in
    // alcun gruppo sono mutuamente esclusivi.
  }
]

```

```
    }
  ],

  // -- Configurazione strumenti built-in (opzionale) --
  "toolConfig?": {

    "magnifier?": {
      "defaultRadius": "<number>",      // Raggio in pixel della
        lente
      "defaultZoomLevel": "<number>"    // Fattore di zoom (es.
        2.4)
    },

    "uartConnector?": {
      "persistAfterConnection": "<boolean>",
        // Se true, il connettore resta visibile dopo la
        connessione.

      "visibleInSteps": ["<string>"],
        // ID degli step in cui il connettore resta visibile.

      "terminalComponentId?": "<string>",
        // Terminale da avviare automaticamente dopo connessione
        UART.

      "terminalDefaultTab?": "<string>"
        // Tab del terminale da attivare (es. "uart").
    },

    "terminal?": {
      "requiresUart": "<boolean>",
      "persistent": "<boolean>",
      "bootStageConditions": [
        {
          "bootStageId": "<string>",
          "unlockedFlags": ["<string>"],
          "hint": "<string>"
        }
      ]
    },

    "firmwareDump?": {
      "probes": [
        {
          "id": "<string>",
```

```

        "role": "<SpiRole>",          // "vcc" | "gnd" | "cs" | "
            clk" | "mosi" | "miso"
        "label": "<string>",
        "color": "<string>"          // Colore hex del filo nel
            simulatore
    }
],
"requiredConnections?": [
    { "probeId": "<string>", "pinId": "<string>" }
],
// Mapping esplicito sonda->pin. Deprecato: la validazione
// avviene ora per ruolo (probe.role == pin.role).

"filePath": "<string>",            // Percorso del file
    firmware
"fileName": "<string>",            // Nome file mostrato nel
    download
"dumpDurationSec": "<number>",    // Durata progress bar in
    secondi

"terminalComponentId?": "<string>",
// Terminale da avviare dopo il completamento del dump.

"terminalDefaultTab?": "<string>"
}
},

// -- Dialogo di completamento (opzionale) --
"completionDialog?": {
    "title": "<string>",
    "description": "<string>",

    "redirectUrl": "<string>",      // URL per pulsante di
        redirect
    "redirectLabel": "<string>",    // Etichetta del
        pulsante

    "downloadFilePath": "<string>", // Percorso file
        scaricabile
    "downloadLabel": "<string>",    // Etichetta pulsante
        download
    "downloadFileName": "<string>", // Nome file per l'
        utente

```



```

    "showCopyFlag": "<boolean>"           // Mostra pulsante copia
      flag
  }
}
```

A.2 Tipi enumerati

Per completezza, si riportano i tipi enumerati utilizzati nello schema.

A.2.1 ObjectiveType

Valore	Descrizione
component	Completamento tramite click su un componente del PCB
pin	Completamento tramite connessione di sonde a pin specifici
uart	Completamento tramite connessione UART corretta (crossover TX/RX)
terminal	Completamento tramite scoperta di flag nel terminale simulato
firmware-dump	Completamento tramite dump firmware SPI

Tabella A.1: Valori ammessi per ObjectiveType.

A.2.2 Tool

Valore	Descrizione
pointer	Cursore standard per interazione con il PCB
magnifier	Lente di ingrandimento per ispezione visiva
multimeter	Multimetro digitale (tensione e resistenza)
probes	Adattatore UART (TX, RX, GND)
firmware-dump	Programmatore SPI per estrazione firmware
custom	Strumento personalizzato definito dall'autore

Tabella A.2: Valori ammessi per Tool.

A.2.3 UartRole

Valore	Tensione tipica	Resistenza tipica
tx	3.3 V	50 Ω
rx	3.3 V	10 k Ω
gnd	0 V	0 Ω
vcc	5.0 V	0 Ω

Tabella A.3: Valori ammessi per `UartRole` con tensione e resistenza tipiche.

A.2.4 SpiRole

Valore	Descrizione	Colore nel simulatore
vcc	Alimentazione	#EF4444 (rosso)
gnd	Massa	#6B7280 (grigio)
cs	Chip Select	#A855F7 (viola)
clk	Clock	#3B82F6 (blu)
mosi	Master Out, Slave In	#F59E0B (ambra)
miso	Master In, Slave Out	#10B981 (verde)

Tabella A.4: Valori ammessi per `SpiRole` con colori associati nel simulatore.

A.3 Sistema di coordinate

Tutte le coordinate nel JSON dell'esercizio utilizzano il **sistema normalizzato a percentuale** descritto nel Capitolo 3 (§3.3.3). Il formato è un array di quattro numeri:

```
[x%, y%, width%, height%]
```

dove `x%` e `y%` rappresentano la posizione dell'angolo superiore sinistro dell'area rispetto alle dimensioni del contenitore, e `width%` e `height%` ne definiscono le dimensioni. La conversione in pixel avviene a runtime nel componente `PCBViewer` in base alle dimensioni effettive del contenitore.

Per obiettivi non spaziali (tipo `terminal`, `uart`), le coordinate sono convenzionalmente impostate a `[0, 0, 0, 0]`.

A.4 Esempio completo

Si riporta un estratto semplificato della configurazione dell'esercizio del caso di studio (Capitolo 5), con due step: identificazione dei componenti e connessione UART.

```
{
  "pcbImage": "/images/preset-ctf-1.png",
  "initialFlag": "flag{????????????????????}",
  "steps": [
    {
      "id": "step-1",
      "title": "Hardware Analysis",
      "description": "Identificare i componenti principali sulla board...",
      "expectedFlag": "flag{CPU-123EPROMDUMP}",
      "availableTools": ["pointer", "magnifier", "multimeter", "firmware-dump"],
      "objectives": [
        {
          "id": "obj-cpu",
          "name": "CPU",
          "type": "component",
          "componentId": "comp-cpu",
          "instruction": "Identifica il System-on-Chip QCA953x ...",
          "hint": "Ha una forma quadrata, e' il chip piu' grande .",
          "flagPart": "CPU-123",
          "coords": [32.07, 49.91, 10.98, 16.11]
        },
        {
          "id": "obj-eprom",
          "name": "EPROM",
          "type": "component",
          "componentId": "comp-eprom",
          "instruction": "Individua la memoria EPROM...",
          "hint": "Ha una forma rettangolare.",
          "flagPart": "EPROM",
          "coords": [60.43, 51.07, 11.11, 28.24]
        },
        {
          "id": "obj-dump",
          "name": "Firmware Dump",
          "type": "pin",
```

```

        "instruction": "Collega le sonde del firmware dumper
        ...",
        "hint": "GND e VCC devono corrispondere ai pin
        indicati.",
        "flagPart": "DUMP",
        "coords": [0, 0, 0, 0],
        "pinConditions": [
            { "pinId": "pin-m-2", "terminal": "fw-probe:fw-probe
            -vcc" },
            { "pinId": "pin-m-1", "terminal": "fw-probe:fw-probe
            -gnd" }
        ],
        "pinLogic": "AND"
    }
]
},
{
    "id": "step-2",
    "title": "UART Connection",
    "description": "Collegare l'adattatore UART ai pin
    corretti...",
    "expectedFlag": "flag{PIN-M-1,_PIN-M-2,_PIN-M-3}",
    "availableTools": ["magnifier", "multimeter", "probes"],
    "objectives": [
        {
            "id": "obj-uart",
            "name": "Connessione UART",
            "type": "pin",
            "instruction": "Collega TX->RX, RX->TX, GND->GND...",
            "hint": "Ricorda il crossover.",
            "flagPart": "PIN-M-1,_PIN-M-2,_PIN-M-3",
            "coords": [0, 0, 0, 0],
            "pinConditions": [
                { "pinId": "pin-m-1", "terminal": "adapter-rx" },
                { "pinId": "pin-m-2", "terminal": "adapter-tx" },
                { "pinId": "pin-m-3", "terminal": "adapter-gnd" }
            ],
            "pinLogic": "AND"
        }
    ]
}
],
"components": [
    {
        "id": "comp-cpu",

```

```

    "name": "CPU",
    "instruction": "",
    "hint": "",
    "flagPart": "CPU",
    "coords": [32.07, 49.91, 10.98, 16.11]
  },
  {
    "id": "comp-eprom",
    "name": "EPROM",
    "instruction": "",
    "hint": "",
    "flagPart": "EPROM",
    "coords": [60.43, 51.07, 11.11, 28.24]
  }
],
"pins": [
  { "id": "pin-m-1", "valueV": 3.3, "valueOhm": 89, "coords":
    [3.06, 42.93, 2, 2] },
  { "id": "pin-m-2", "valueV": 0, "valueOhm": 0, "coords":
    [2.94, 46.25, 2, 2] },
  { "id": "pin-m-3", "valueV": 0, "valueOhm": 0, "coords":
    [2.94, 50.07, 2, 2] }
],
"uartPins": [],
"toolConfig": {
  "magnifier": { "defaultRadius": 90, "defaultZoomLevel": 2.4
  },
  "uartConnector": {
    "persistAfterConnection": true,
    "visibleInSteps": ["step-2"],
    "terminalComponentId": "tc-terminal-1"
  },
  "firmwareDump": {
    "probes": [
      { "id": "fw-probe-gnd", "role": "gnd", "label": "GND", "
        color": "#6B7280" },
      { "id": "fw-probe-vcc", "role": "vcc", "label": "VCC", "
        color": "#EF4444" }
    ],
    "filePath": "/uploads/firmware-fw_test.bin",
    "fileName": "fw_test.bin",
    "dumpDurationSec": 1,
    "terminalComponentId": "tc-terminal-1"
  }
},

```

```
"completionDialog": {  
  "title": "Exercise Completed!",  
  "description": "Congratulations, you have successfully  
    completed all objectives.",  
  "redirectUrl": "",  
  "redirectLabel": "Next Exercise",  
  "downloadFilePath": "",  
  "downloadLabel": "Download File",  
  "downloadFileName": "",  
  "showCopyFlag": true  
}  
}
```

Appendice B

Schema della configurazione terminale

Si riporta lo schema completo della struttura dati `TerminalConfig`, che definisce il comportamento del terminale simulato nella piattaforma PCB-CTF. Lo schema governa l'intero sistema: tab, comandi, filesystem, sequenza di boot, flag e vincoli di esecuzione.

Nel formato di persistenza, la configurazione terminale è incapsulata in un array di **componenti terminale**, ciascuno con un proprio identificatore, nome e configurazione indipendente:

```
[
  {
    "id": "<string>",           // Identificatore univoco del
                                componente
    "name": "<string>",         // Nome descrittivo (es. "UART
                                Console")
    "config": { /* TerminalConfig */ }
  }
]
```

Questa struttura modulare consente di definire più terminali indipendenti nello stesso esercizio (ad esempio, una console UART e una macchina locale), ciascuno con il proprio filesystem, comandi e sequenza di boot.

B.1 Struttura principale (`TerminalConfig`)

```
{
  "metadata": {
```

```

    "version": "<string>",           // Versione dello schema (es
        . "1.0.0")
    "name": "<string>",              // Nome della configurazione
    "description": "<string>",       // Descrizione testuale
    "author?": "<string>"           // Autore della
        configurazione
},

"tabs": [ /* TabConfig[] -- vedi B.2 */ ],

"flags?": { /* FlagSystem -- vedi B.5 */ },

"globalCommands?": {
    // Comandi disponibili in tutti i tab. Stessa struttura di
    // CommandDefinition (B.3). Utile per comandi trasversali
    // come "help" o "exit".
    "<commandName>": { /* CommandDefinition */ }
},

"customFunctions?": {
    // Funzioni JavaScript custom referenziabili dai generatori
    // di output dinamico e dagli effetti collaterali.
    "<functionName>": "<string>"    // Codice della funzione
}
}

```

B.2 Configurazione del tab (TabConfig)

Ogni tab rappresenta un ambiente terminale indipendente, con il proprio filesystem, set di comandi, variabili d'ambiente e sequenza di boot opzionale.

```

{
    "id": "<string>",                // Identificatore
        univoco del tab
    "name": "<string>",              // Nome visualizzato (es
        . "UART Console")
    "initialPath?": "<string>",      // Directory iniziale (
        default: "/" )

    "filesystem": { /* FilesystemStructure -- vedi B.4 */ },

    "commands": {
        // Dizionario dei comandi disponibili nel tab.
    }
}

```



```

    // La chiave e' il nome del comando; il valore e' un
    // CommandDefinition.
    "<commandName>": { /* CommandDefinition -- vedi B.3 */ }
},

"bootSequence?": {
    // Sequenza di boot animata, riprodotta all'apertura del
    // terminale.
    "initialStage": "<string>",           // ID del primo stadio
    "stages": [
        {
            "id": "<string>",              // Identificatore dello
            // stadio
            "name": "<string>",            // Nome descrittivo
            "lines": ["<string>"],         // Righe di testo animate
            "duration?": "<number>",       // Durata in millisecondi
            "nextStage?": "<string>",       // Stadio successivo (
            // auto-transizione)
            "prompt?": "<string>"          // Prompt dello stadio (
            // se interattivo)
        }
    ]
},

"environment?": {
    // Variabili d'ambiente del tab. Accessibili dai comandi
    // template.
    "USER": "<string>",
    "HOME": "<string>",
    "PATH": "<string>",
    "SHELL": "<string>"
    // ... qualsiasi coppia chiave-valore
},

"defaultConstraints?": { /* CommandConstraints -- vedi B.3.1
    */ }
// Vincoli applicati a tutti i comandi del tab.
// I vincoli specificati a livello di singolo comando hanno
// precedenza su quelli di default.
}

```

B.2.1 Stadi speciali della sequenza di boot

Il terminale riconosce tre identificatori di stadio con comportamento hardcoded:

ID stadio	Comportamento
uboot_wait	Mostra un messaggio “Hit any key to stop autoboot” con countdown. Se lo studente preme un tasto, transita allo stadio successivo (tipicamente uboot_shell); altrimenti, il boot prosegue automaticamente.
login	Mostra un prompt di login. L’input dello studente viene trattato come nome utente e inoltrato al sistema di comandi per la validazione.
password	Mostra un prompt di password (input mascherato). L’input viene inoltrato al sistema di comandi; il comando corrispondente alla password corretta deve produrre la transizione allo stadio shell.

B.3 Definizione di un comando (CommandDefinition)

```
{
  "name?": "<string>",
  // Nome del comando. Opzionale: se omissso, viene derivato
  // dalla
  // chiave nel dizionario commands.

  "aliases?": [<string>],
  // Nomi alternativi che invocano lo stesso comando.
  // Es. ["help"] come alias di "?".

  "description": "<string>",
  // Descrizione del comando (usata internamente e nell'
  // eventuale help).

  "usage?": "<string>",
  // Stringa di utilizzo (es. "cat <file>").

  "handler": "<'builtin' | 'custom' | 'script' | 'dynamic'>",
  // Tipo di handler che gestisce l'esecuzione:
  // "builtin" -- implementazione nativa in JavaScript (ls,
  // cd, cat, etc.)
  // "custom" -- output definito nella configurazione
  // "script" -- esecuzione di una funzione JavaScript
  // custom
  // "dynamic" -- generazione dinamica tramite funzione
  // registrata

  "builtinType?": "<string>",
  // Per handler="builtin": nome dell'implementazione nativa.
```

```
// Valori supportati: "ls", "cd", "cat", "pwd", "grep", "find", "clear"

"constraints?": { /* CommandConstraints -- vedi B.3.1 */ },

"output?": { /* CommandOutput -- vedi B.3.2 */ },

"sideEffects?": {
  // Effetti collaterali eseguiti dopo la produzione dell'
  // output.

  "unlockFlags?": [
    // Flag da sbloccare. Può essere:
    //   - una stringa (ID del flag, sbloccato
    //     incondizionatamente)
    //   - un oggetto FlagUnlock con condizione

    "<string>",
    // oppure:
    {
      "id": "<string>", // ID del flag da
        sbloccare
      "condition?": { /* ConditionCheck -- vedi B.3.3 */ }
      // Se presente, il flag viene sbloccato solo se la
      // condizione
      // e' soddisfatta (es. argomento specifico passato al
      // comando).
    }
  ],

  "setState?": {
    // Modifica variabili di stato del terminale.
    // Uso tipico: transizione dello stadio di boot.
    "bootStage": "<string>",
    // ... qualsiasi coppia chiave-valore
  },

  "executeCommand?": "<string>",
  // Comando da eseguire automaticamente dopo quello corrente.

  "customEffect?": "<string>"
  // Nome di una funzione custom da invocare.
},

"help?": "<string>"
```

```
// Testo di aiuto esteso.  
}
```

B.3.1 Vincoli del comando (CommandConstraints)

I vincoli determinano le condizioni sotto le quali un comando può essere eseguito. Se un vincolo non è soddisfatto, il comando produce un messaggio di errore configurabile.

```
{  
  "path?": {  
    // Vincolo sul percorso corrente del filesystem.  
    "type": "<'exact' | 'contains' | 'startsWith' | 'regex'>",  
    "value": "<string | string[]>", // Valore o lista di  
      valori ammessi  
    "errorMessage": "<string>"  
  },  
  
  "permissions?": {  
    // Vincolo sui permessi dell'utente.  
    "requireRoot?": "<boolean>", // Richiede utente root  
    "requireUser?": "<string>", // Richiede un utente  
      specifico  
    "errorMessage": "<string>"  
  },  
  
  "prerequisites?": {  
    // Vincolo su prerequisiti (comandi eseguiti, file esistenti  
      , flag).  
    "commands?": [<string>], // Comandi che devono  
      essere stati eseguiti  
    "files?": [<string>], // File che devono  
      esistere  
    "flags?": [<string>], // Flag che devono  
      essere state scoperte  
    "condition?": "<'all' | 'any'>", // Logica di  
      combinazione (default: "all")  
    "errorMessage": "<string>"  
  },  
  
  "arguments?": {  
    // Vincolo sugli argomenti passati al comando.  
  }
```

```

    "min?": "<number>",           // Numero minimo di
        argomenti
    "max?": "<number>",           // Numero massimo di
        argomenti
    "required?": [
        {
            "index": "<number>",    // Posizione dell'
                argomento (0-based)
            "pattern?": "<string>",    // Pattern regex che
                deve matchare
            "errorMessage?": "<string>"
        }
    ],
    "errorMessage?": "<string>"
},

"state?": {
    // Vincolo sullo stato del terminale.
    "bootStage?": "<string | string[]>",
    // Stadio (o stadi) di boot in cui il comando e' disponibile
    .
    // Uso tipico: limitare comandi U-Boot allo stadio "
        uboot_shell",
    // comandi filesystem allo stadio "shell".

    "variables?": {
        // Variabili di stato che devono avere valori specifici.
        "<key>": "<any>"
    },
    "errorMessage?": "<string>"
}
}

```

B.3.2 Tipi di output (CommandOutput)

Il sistema supporta sei tipi di output, ciascuno con semantica e struttura propria.

Output statico

```

{
    "type": "static",
    "lines": ["<string>"]
    // Array di righe restituite verbatim.
}

```

```
}
```

Output condizionale

```
{
  "type": "conditional",
  "conditions": [
    {
      "if": { /* ConditionCheck */ },
      "then": { /* CommandOutput */ },
      "else?": { /* CommandOutput */ }
    }
  ],
  "default?": { /* CommandOutput */ }
  // Valutazione sequenziale: la prima condizione soddisfatta
  // determina l'output. Se nessuna e' vera, si usa default.
}
```

Output template

```
{
  "type": "template",
  "template": "<string>",
  // Stringa con placeholder {{variabile}}.

  "variables": {
    "<name>": {
      "type": "<'static' | 'computed' | 'random' | 'state'>",
      "value?": "<any>", // Per type="static"
      "min?": "<number>", // Per type="random":
        limite inferiore
      "max?": "<number>", // Per type="random":
        limite superiore
      "options?": ["<string>"], // Per type="random":
        scelta tra opzioni
      "compute?": "<string>" // Per type="computed":
        nome funzione
    }
  }
}
```

Output lookup

```
{
  "type": "lookup",
  "argIndex": "<number>",
  // Indice dell'argomento da confrontare (0-based).

  "matchType": "<'contains' | 'equals' | 'regex'>",
  // Strategia di matching.

  "table": {
    // Mappa valore -> output. La chiave e' il valore da
    // matchare,
    // il valore e' l'array di righe da restituire.
    "<matchValue>": ["<string>"]
  },

  "default?": { /* CommandOutput */ }
  // Output di fallback se nessun match.
}
```

Output dinamico

```
{
  "type": "dynamic",
  "generator": "<string>",      // Nome della funzione generatore
  "args?": ["<any>"]           // Argomenti opzionali
}
```

Output script

```
{
  "type": "script",
  "script": "<string>"          // Nome funzione o script inline
}
```

B.3.3 Controllo delle condizioni (ConditionCheck)

Le condizioni sono utilizzate sia negli output condizionali sia nello sblocco condizionale delle flag.

```

{
  "type": "<'argument' | 'path' | 'file' | 'flag' | 'state' | 'custom'>",

  // Per type="argument":
  "index?": "<number>",           // Indice dell'argomento (0-based)
  "contains?": "<string>",         // L'argomento contiene la stringa
  "equals?": "<string>",           // L'argomento e' esattamente uguale
  "regex?": "<string>",            // L'argomento matcha il pattern regex

  // Per type="file":
  "exists?": "<boolean>",          // Il file esiste/non esiste

  // Per type="state":
  "variable?": "<string>",         // Nome della variabile di stato
  "value?": "<any>",               // Valore atteso

  // Per type="custom":
  "check?": "<string>"             // Nome della funzione di verifica
}

```

B.4 Struttura del filesystem (FilesystemStructure)

Il filesystem del terminale supporta due formati di definizione: un formato ad **albero annidato** (più leggibile per l'autore) e un formato **piatto** (utilizzato internamente dal sistema). Il `TerminalConfigLoader` converte automaticamente il formato annidato in formato piatto al caricamento.

```

{
  "tree?": {
    // Formato annidato: le chiavi sono nomi di file/directory.
    // - Se il valore e' una stringa -> file con quel contenuto
    // - Se il valore e' un oggetto -> directory (ricorsiva)
    // - Se il valore e' un oggetto vuoto {} -> directory vuota

    "etc": {                       // directory

```



```

    "passwd": "root:x:0:0:root:/root:/bin/sh",    // file
    "rc.d": {                                     // sotto-
        directory
        "rcS": "#!/bin/sh\nmount -a\n..."      // file
    }
},
"tmp": {}                                       // directory
    vuota
},

"directories": {
    // Formato piatto: percorso -> lista di entry contenute.
    // Popolato automaticamente dal loader a partire da tree.
    "/": ["etc", "tmp"],
    "/etc": ["passwd", "rc.d"],
    "/etc/rc.d": ["rcS"]
},

"files": {
    // Formato piatto: percorso completo -> contenuto o metadati
    .
    // Può essere una stringa (shorthand) o un oggetto FileNode
    .

    "/etc/passwd": "root:x:0:0:root:/root:/bin/sh",
    // oppure:
    "/etc/passwd": {
        "name": "passwd",
        "type": "file",
        "content": "root:x:0:0:root:/root:/bin/sh",
        "permissions?": "-rw-r--r--",
        "owner?": "root",
        "size?": 37,
        "modified?": "Jun 16 2015",
        "metadata?": {}
    }
},

"fileDefaults?": {
    // Valori di default per permessi e proprietario, applicati
    // a tutti i file che non specificano valori propri.
    "permissions?": "<string>",           // Es. "-rw-r--r--"
    "owner?": "<string>"                   // Es. "root"
}
}

```

B.4.1 Processo di normalizzazione

Il `TerminalConfigLoader` esegue la normalizzazione del filesystem in quattro fasi:

1. **Flatten** — converte il `tree` annidato in strutture piatte `directories` e `files`;
2. **Normalize** — assicura che tutte le directory intermedie siano presenti e che la root `/` esista;
3. **Compute** — calcola le dimensioni dei file e verifica la coerenza dei percorsi;
4. **Build** — genera la struttura finale pronta per l'uso da parte dei comandi built-in.

B.5 Sistema di flag (FlagSystem)

```
{
  "parts": [
    {
      "id": "<string>",
      // Identificatore univoco del frammento di flag.
      // Referenziato nei sideEffects.unlockFlags dei comandi.

      "part": "<string>",
      // Testo del frammento. Contribuisce alla costruzione
      // progressiva della flag visibile allo studente.

      "description": "<string>",
      // Descrizione del frammento (visibile nel pannello di
      // progressione).

      "hint": "<string>"
      // Suggerimento per lo studente.
    }
  ],

  "completeFlag": "<string>",
  // Flag finale completa, risultante dalla concatenazione di
  // tutti i frammenti. Esempio: "flag{
  //   b00t_r00t_h4sh_l34k_1nj3ct_sh3ll}"

  "showProgress?": "<boolean>"
  // Se true, mostra la barra di progressione con i frammenti
  // scoperti e quelli ancora nascosti.
```

```
}
```

B.6 Contesto di esecuzione (CommandContext)

Per completezza, si riporta la struttura del contesto passato a ogni esecuzione di comando. Questa interfaccia non è configurabile dall'autore, ma è costruita automaticamente dal sistema a runtime.

```
{
  "command": "<string>",           // Nome del comando eseguito
  "args": ["<string>"],           // Argomenti passati
  "currentPath": "<string>",       // Directory corrente
  "filesystem": { /* FilesystemStructure */ },
  "environment": { /* Record<string, string> */ },
  "bootStage": "<string>",        // Stadio di boot corrente
  "discoveredFlags": ["<string>"], // Flag già scoperte
  "history": ["<string>"],        // Storico dei comandi
                                   eseguiti
  "state": { /* Record<string, any> */ }, // Variabili di stato
  "tab": "<string>"               // ID del tab corrente
}
```

B.7 Risultato dell'esecuzione (CommandExecutionResult)

Analogamente, la struttura del risultato restituito dall'esecutore al termine di ogni comando:

```
{
  "success": "<boolean>",         // Esecuzione riuscita
  "output": ["<string>"],         // Righe di output
  "error?": "<string>",           // Messaggio di errore (se
                                   fallita)
  "sideEffects?": {
    "newFlags?": ["<string>"],    // Flag sbloccate dal
                                   comando
    "stateChanges?": {},          // Variabili di stato
                                   modificate
    "stageChange?": "<string>",   // Nuovo stadio di boot
    "pathChange?": "<string>"    // Nuova directory corrente
  }
}
```

B.8 Esempio: configurazione del tab UART Console

Si riporta un estratto semplificato della configurazione del tab “UART Console” dal caso di studio, che illustra l’uso combinato di comandi built-in, comandi custom con vincoli di boot stage, output lookup con sblocco condizionale di flag, e sequenza di boot multi-stadio.

```
{
  "id": "uart",
  "name": "UART Console",
  "initialPath": "/",
  "filesystem": {
    "tree": {
      "etc": {
        "passwd": "root:x:0:0:root:/root:/bin/sh",
        "shadow": "root:$1$GTN.gpri$DlSyKvZKMR9A9Uj9e9wR3
                  /:15771:0:99999:7:::"
      },
      "proc": {
        "cpuinfo": "system type   : QCA953x\nprocessor   : 0\ncpu
                  model      : MIPS 24Kc V7.4"
      }
    },
    "directories": {},
    "files": {},
    "fileDefaults": { "permissions": "-rw-r--r--", "owner": "
                      root" }
  },
  "commands": {
    "ls": { "handler": "builtin", "builtinType": "ls",
            "description": "List directory contents" },
    "cat": { "handler": "builtin", "builtinType": "cat",
            "description": "Display file contents",
            "constraints": { "arguments": { "min": 1 } } },
    "printenv": {
      "description": "Print environment variables",
      "handler": "custom",
      "constraints": { "state": { "bootStage": "uboot_shell" } }
    },
    ,
  },
}
```

```

    "output": {
      "type": "static",
      "lines": [
        "bootargs=console=ttyS0,115200 root=31:2 rootfstype=
          jffs2 init=/sbin/init",
        "bootcmd=bootm 0x9f020000",
        "bootdelay=1",
        "baudrate=115200"
      ]
    },
    "sideEffects": { "unlockFlags": ["boot"] }
  },
  "strings": {
    "description": "Print printable characters in files",
    "handler": "custom",
    "constraints": { "arguments": { "min": 1 } },
    "output": {
      "type": "lookup",
      "argIndex": 0,
      "matchType": "contains",
      "table": {
        "mtdblock3": ["TP-LINK_807C", "WPA2-PSK",
          "MyW1F1P@ssw0rd!", "192.168.0.1"],
        "httpd": ["execFormatCmd", "system", "popen"],
        "backdoorTest": ["socket", "connect", "/bin/sh",
          "192.168.0.100", "4444"]
      }
    }
  },
  "sideEffects": {
    "unlockFlags": [
      { "id": "leak",
        "condition": { "type": "argument", "index": 0,
          "contains": "mtdblock3" } },
      { "id": "inject",
        "condition": { "type": "argument", "index": 0,
          "contains": "httpd" } },
      { "id": "shell",
        "condition": { "type": "argument", "index": 0,
          "contains": "backdoorTest" } }
    ]
  }
},
"root": {
  "description": "Login as root",
  "handler": "custom",

```

```

        "constraints": { "state": { "bootStage": "login" } },
        "sideEffects": {
            "setState": { "bootStage": "password", "loginUser": "
                root" }
        }
    },
    "sohoadmin": {
        "description": "Enter password",
        "handler": "custom",
        "constraints": { "state": { "bootStage": "password" } },
        "output": {
            "type": "static",
            "lines": [
                "", "BusyBox v1.01 Built-in shell (ash)",
                "Enter 'help' for a list of built-in commands.",
                "", ""
            ]
        },
        "sideEffects": {
            "unlockFlags": ["root"],
            "setState": { "bootStage": "shell", "currentPath": "/" }
        }
    }
},
"bootSequence": {
    "initialStage": "connecting",
    "stages": [
        { "id": "connecting", "name": "Connecting",
          "lines": ["Connecting to UART interface..."],
          "duration": 1500, "nextStage": "booting" },
        { "id": "booting", "name": "Bootting",
          "lines": [
              "U-Boot 1.1.4 (Jun 16 2015)",
              "DRAM: 32 MB", "Flash: 4 MB"
            ],
          "duration": 2000, "nextStage": "uboot_wait" },
        { "id": "uboot_wait", "name": "U-Boot Wait",
          "lines": ["Hit any key to stop autoboot: 3"],
          "duration": 1000, "nextStage": "uboot_shell",
          "prompt": "Press Enter to access U-Boot console" },
        { "id": "uboot_shell", "name": "U-Boot Shell",
          "lines": ["U-Boot> "], "prompt": "U-Boot> " },
        { "id": "kernel_boot", "name": "Kernel Boot",
          "lines": [
              "Bootting QCA953x", "Linux version 2.6.31",
              "VFS: Mounted root (squashfs) readonly"
            ],
          "duration": 2500, "nextStage": "login" },
        { "id": "login", "name": "Login",
          "lines": ["TP-LINK login: "], "prompt": "Login: " },
        { "id": "password", "name": "Password",

```

```
        "lines": [], "prompt": "Password: " },
    { "id": "shell", "name": "Shell",
      "lines": ["BusyBox v1.01 Built-in shell (ash)"],
      "prompt": "# " }
  ]
},
"environment": {
  "USER": "root", "HOME": "/root",
  "PATH": "/bin:/sbin:/usr/bin:/usr/sbin", "SHELL": "/bin/sh"
},
"defaultConstraints": { "state": { "bootStage": "shell" } }
}
```

Questo esempio illustra i principali pattern di configurazione:

- I comandi `ls` e `cat` utilizzano handler built-in con vincoli sugli argomenti;
- Il comando `printenv` è limitato allo stadio `uboot_shell` e sblocca incondizionatamente la flag `boot`;
- Il comando `strings` usa un output di tipo lookup con sblocco condizionale: la flag sbloccata dipende dall'argomento passato dallo studente;
- I comandi `root` e `soheadmin` gestiscono il flusso di login tramite transizioni di stato (`bootStage`);
- I `defaultConstraints` del tab limitano tutti i comandi (tranne quelli con vincoli propri) allo stadio `shell`, impedendo l'uso di comandi filesystem durante il prompt U-Boot.